BlindSpy

Classifying encrypted traffic (ISCXVPN2016) from 23 flow timing features

Report at milestone v0.2-tuned

Built 2026-10-29 from the report drafts at commit e1f81ac

1 Introduction

1.1 Problem

Most network traffic is now encrypted. Deep packet inspection, which classifies traffic by reading packet contents, no longer works when the contents are hidden. Network operators still need to know what kind of traffic a connection carries (for quality of service, access control and security monitoring) and whether it is routed through a VPN.

Applications leave a timing pattern even when their contents are encrypted. A voice call sends small packets at a steady rhythm, video streaming arrives in bursts, and a file transfer runs at full speed with almost no idle time. BlindSpy trains classifiers on these patterns only: 23 flow timing statistics per connection, never the payload.

We use the ISCXVPN2016 dataset [2] published with Draper-Gil et al. [1]. Each row is one flow described by 23 time-related features and a label. The dataset is provided in four versions, built with flow timeouts of 15, 30, 60 and 120 seconds. The timeout is a setting of the flow builder (it decides when an inactive flow is closed); it is not the duration of each flow.

1.2 Tasks

We follow the three tasks of the original paper [1], each scored on its own:

Task	Description	Scored by
A1	One binary classifier: VPN vs non-VPN	Precision and recall per class, macro F1, ROC-AUC
A2	Two separate 7-class classifiers, one trained on VPN flows and one on non-VPN flows	Each model scored separately; never merged into one score
B	One 14-class classifier over VPN and non-VPN traffic together	Macro F1 and per-class scores over all 14 labels

The seven traffic types are browsing, email, chat, streaming, file transfer, VoIP and P2P; task B uses each type twice (VPN and non-VPN).

1.3 What BlindSpy can and cannot claim

- It predicts the traffic type from aggregate timing statistics and never reads packet contents.
- It does not prove that traffic is encrypted.
- The data was captured in 2016, in a lab, with specific applications and one VPN setup. Results may not hold for current applications, VPN providers or networks. The conclusion repeats this.

1.4 Related work

Study	Data and setup	Models	Reported result
Draper-Gil et al. 2016 [1]	ISCXVPN2016 ARFF files, tasks A1, A2, B, each timeout separately; Weka, 10-fold cross-validation	C4.5, KNN	A1 at 15 s, C4.5: precision 0.89 (VPN) and 0.906 (non-VPN); KNN: 0.848 and 0.846. A2 at 15 s, C4.5: average precision 0.84 (VPN model), 0.89 (non-VPN model). B, best timeout: average precision 0.783 (C4.5), 0.711 (KNN)
Bagui et al. 2017 [12]	Same time-related features	Random Forest, gradient boosting and others	About 90% (cited from the public abstract only; the full paper was not available to us, so no further detail is claimed)
Jorgensen et al. 2023 [14]	ISCXVPN2016 captures	Uncertainty-aware traffic labelling	Documents quality problems in the dataset's captures (see Section 7)
Isnan et al. 2026 [13]	Kaggle copy of the dataset, 59,706 rows; 18,719 duplicates removed, leaving 40,987; 14 classes; 80/20 split	Random Forest, 300 trees	88.01% accuracy

Paper values are taken from `reports/paper_baseline.csv`, which records each number's page and whether it is stated in the text or read from a figure.

Why published scores (80–99%) are not directly comparable. Studies on this dataset differ in the split (k-fold vs a single hold-out), the class setup (binary, 7-class or 14-class), whether duplicates are removed, and

whether flows from different timeout versions are mixed. Each of these can move accuracy by several points, so we compare only under matched protocols.

The Isnan et al. data. The Kaggle file used by Isnan et al. is the eight A2 files (VPN and non-VPN at all four timeouts) stacked together: compared as a multiset, its 59,706 rows equal the rows of those files (5,151 + 5,631 + 8,965 + 9,793 + 6,917 + 7,734 + 8,580 + 6,935), with no row left over and the same labels. It has 40,987 unique rows, exactly their count. 7,093 distinct flows appear in two to four timeout files, while only 3,676 rows are duplicates within one file (E00). So most of their 18,719 "duplicates" are the same flows repeated across timeout versions, and a random 80/20 split of the stacked file mixes the four flow-building settings in both training and test data. E16 reproduces their 88.01% (0.8796 on the same split) and shows that with the timeouts kept apart the same method scores 0.82-0.87 (mean 0.8430). A size control in E16 attributes that drop to training-set size, not to the mixing itself: stacking the four files gives 2.3-4.3 times as many training rows, and at equal size the mixed setup scores no higher (0.8395). Their score is therefore not comparable with a single-timeout score. The file also uses the A2 feature set (`total_fiat` and `total_biat` in place of the paper's `std_fiat` and `std_biat`; Section 2).

1.5 Contributions

1. A reproduction of the paper's C4.5 and KNN results under its own 10-fold protocol, compared precision with precision and recall with recall. [E02, E03, paper benchmark]
2. A leakage-safe protocol (stratified 70/15/15 split, cross-validation inside the training set only, test set scored once) applied to all three tasks and all four timeouts.
3. Stronger models (Random Forest, XGBoost, LightGBM) and a fair comparison of seven method types: every type tuned with the same search budget, plus class balancing and a two-step model for task B. [E04–E08, E10, E17]
4. A reproduction of a recent published result on a copy of this dataset (Isnan et al.), then a corrected version that keeps the timeout files apart, with a control that separates the effect of training-set size from the effect of mixing timeouts. [E16]
5. A check of the dataset itself: whether the ARFF column names match the data, duplicates, odd values and the class mix across timeouts, recorded as versioned data with a step-by-step cleaning log. [E00]
6. Analyses of what the models learned and how far the results can be trusted: feature-group ablation, SHAP, learning curve, split-seed stability and calibration. [E09, E13–E15]
7. A full history: every run, data version and experiment, traceable to a commit.

2 Dataset

2.1 Source and format

We use ISCXVPN2016 [2], published with Draper-Gil et al. [1] by the Canadian Institute for Cybersecurity, University of New Brunswick. The traffic was captured in 2016 in a lab, with and without a VPN, for seven traffic types: browsing, chat, streaming, mail, VoIP, P2P and file transfer. The authors turned the captures into flows with ISCXFlowMeter and published the flows as 20 ARFF files (Weka's text format). Each row is one flow: 23 numeric time-related features (times in microseconds, rates per second) and a label. We use only these files. They hold no payload, addresses, ports or timestamps. The SHA-256 hash of every file is recorded in `data/arff_sha256.txt`.

Each scenario is published at four flow timeouts (15, 30, 60 and 120 s). The timeout is a setting of the flow builder, not a flow duration. Flows longer than the timeout exist (up to 646 s) because the timeout closes a flow only after it has been inactive for that long.

2.2 Tasks and files

Task	Files	Labels
A1	`Scenario A1-ARFF`, 4 files	VPN, Non-VPN
A2-VPN	`Scenario A2-ARFF`, 4 `-VPN` files	7 VPN traffic types
A2-NonVPN	`Scenario A2-ARFF`, 4 `-NO-VPN` files	7 traffic types
B	`Scenario B-ARFF`, 4 files	14 classes (7 types x VPN / non-VPN)
(B-AllinOne)	`Scenario B-ARFF`, 4 `-AllinOne` files	7 types, VPN and non-VPN merged; inventoried, not a task

file	task	timeout	header	rows	classes	core_task
Scenario A1-ARFF/TimeBasedFeatures-Dataset-15s-VPN.arff	A1	15	T	18,758	2	1
Scenario A1-ARFF/TimeBasedFeatures-Dataset-30s-VPN.arff	A1	30	T	14,651	2	1
Scenario A1-ARFF/TimeBasedFeatures-Dataset-60s-VPN.arff	A1	60	S	15,515	2	1
Scenario A1-ARFF/TimeBasedFeatures-Dataset-120s-VPN.arff	A1	120	T	10,782	2	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-15s-VPN.arff	A2-VPN	15	T	9,793	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-30s-VPN.arff	A2-VPN	30	T	7,734	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-60s-VPN.arff	A2-VPN	60	T	6,935	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-120s-VPN.arff	A2-VPN	120	T	5,631	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-15s-NO-VPN.arff	A2-NonVPN	15	T	8,965	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-30s-NO-VPN.arff	A2-NonVPN	30	T	6,917	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-60s-NO-VPN.arff	A2-NonVPN	60	T	8,580	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-120s-NO-VPN.arff	A2-NonVPN	120	T	5,151	7	1
Scenario B-ARFF/TimeBasedFeatures-Dataset-15s.arff	B	15	T	18,758	14	1

file	task	timeout	header	rows	classes	core_task
Scenario B-ARFF/TimeBasedFeatures-Dataset-30s.arff	B	30	T	14,651	14	1
Scenario B-ARFF/TimeBasedFeatures-Dataset-60s.arff	B	60	S	15,515	14	1
Scenario B-ARFF/TimeBasedFeatures-Dataset-120s.arff	B	120	T	10,782	14	1
Scenario B-ARFF/TimeBasedFeatures-Dataset-15s-AllinOne.arff	B-AllinOne	15	T	18,758	7	0
Scenario B-ARFF/TimeBasedFeatures-Dataset-30s-AllinOne.arff	B-AllinOne	30	T	14,651	7	0
Scenario B-ARFF/TimeBasedFeatures-Dataset-60s-AllinOne.arff	B-AllinOne	60	S	15,515	7	0
Scenario B-ARFF/TimeBasedFeatures-Dataset-120s-AllinOne.arff	B-AllinOne	120	T	10,782	7	0

Two features of the raw files matter for everything below. A1, A2 and B hold **the same flows** at each timeout: every A1 row matches an A2 row on seven integer features at all four timeouts (100%), and the labels agree key by key (0 disagreements). B and A1 are identical cell for cell except for one corrupt column (Section 2.4). BROWSING and VPN-BROWSING hold exactly 2,500 flows in every raw A2 file, which suggests the dataset authors capped those classes.

2.3 Column-name check

Problem. The first row of A1 15 s has `duration` = 9,368,711 us (9.4 s) but `total_fiat` = 16 and `total_biat` = 4. A total forward gap of 16 us in a 9-second flow is impossible. The paper's Table 2 [1] and the authors' 2016 poster [15] describe the forward and backward inter-arrival features as min, mean, max and std, with no totals. Yet 17 of the 20 headers list `total_fiat` and `total_biat` and no std. The three 60 s files of A1 and B have a second header that matches the paper.

Method. We read every file under both name lists. On every row we tested identities that hold whatever the traffic: min <= mean <= max for each gap statistic, 0 <= std <= max - min, mean flow gap x (packets - 1) = duration, and, for totals, total / mean + 1 per direction = packet count. Means of 10^7 or more are stored to 3 significant figures in the files, so a tolerance of 0.5% is allowed. We also tested the reading that `total_fiat` is a forward packet count.

check	A1 15 s (T)	A1 15 s (S)	A2-VPN 15 s (T)	A2-VPN 15 s (S)	A2-NonVPN 15 s (T)	A2-NonVPN 15 s (S)	B 15 s (T)	B 15 s (S)
fiat: min <= mean <= max	0.0	100.0	100.0	18.8	100.0	5.24	0.0	100.0
biat: min <= mean <= max	0.01	100.0	100.0	14.07	100.0	4.06	0.01	100.0
flowiat: min <= mean <= max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
active: min <= mean <= max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
idle: min <= mean <= max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
mean_flowiat x (packets - 1) = duration (+-1%)	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
bytes per packet in 40-1500	97.72	97.72	99.26	99.26	96.06	96.06	97.72	97.72
min_flowiat <= min(min_fiat, min_biat)	100.0	100.0	100.0	80.98	100.0	83.1	100.0	100.0
total_fiat <= duration	100.0	-	100.0	-	100.0	-	100.0	-
total/mean + 1, fwd + bwd = packets (+-2%)	0.0	-	100.0	-	100.0	-	0.0	-
total_fiat as packet count (+-10%)	1.98	-	0.0	-	0.04	-	1.98	-
0 <= std_fiat <= max - min	-	100.0	-	0.0	-	0.0	-	100.0
0 <= std_biat <= max - min	-	100.0	-	0.0	-	0.0	-	100.0

The same pattern holds at every timeout (`column_name_check` in the manifest has all 16 files).

Result. The headers are wrong for 9 of the 12 A1/B files, and the fault is a shift, not a shuffle. These files hold the paper's features in the order min, max, mean, std for fiat and biat, but their header names them total, min, max, mean. Under the corrected names every identity passes on 100% of rows; under the header names, min <= mean <= max passes on at most 0.02%. A2 files are named correctly and hold a different pair of features: `total_fiat` and `total_biat` (the sum of the forward and backward gaps) instead of `std_fiat` and `std_biat`. The packet-count reading passes on at most 2% of rows in any file, so it is ruled out. `COLUMN_MAP` in `src/blindspy/data.py` renames the 9 shifted files. It maps `total_fiat->min_fiat`, `total_biat->min_biat`, `min_fiat->max_fiat`, `min_biat->max_biat`, `max_fiat->mean_fiat`, `max_biat->mean_biat`, `mean_fiat->std_fiat`, `mean_biat->std_biat`; the other 15 names are unchanged. All feature names in this report (importances, SHAP, ablation) are the corrected ones.

Units and value ranges. Table 2 gives no units. The data does: with packets = flow packets per second x duration / 10^6, the mean flow gap x (packets - 1) equals the duration within 1% on 100% of rows in all 16 files, which holds only if times are in microseconds and rates per second. Every measured gap, active or idle value is at most the flow's duration (100% of values), as Table 2's definitions require. Under the corrected names the A1/B feature list is exactly Table 2's 23 features; the A2 list differs only in `total_fiat` and `total_biat` (`reports/data_checks_v1.json`).

feature	min	median	p99	max	exactly_-1	other_negative	% <= duration
duration	0	14,422,310	119,998,194	646,565,983	0	0	-
min_fiat	-1	112	45,211,908	119,999,708	6,308	0	100.0
min_biat	-1	45	45,213,098	119,999,620	10,522	0	100.0
max_fiat	-1	1,650,890	104,244,407	646,287,731	6,308	0	100.0
max_biat	-1	2,294,484	63,987,999	600,109,667	10,522	0	100.0
mean_fiat	0	158,066	45,200,000	152,000,000	0	0	100.0
mean_biat	0	83,359	45,200,000	120,000,000	0	0	100.0
std_fiat	0	27,808	34,000,000	264,000,000	0	0	-
std_biat	0	24,804	25,700,000	98,000,000	0	0	-
flowPktsPerSecond	0	8.96615	57,143	2,000,000	0	0	-
flowBytesPerSecond	0	1,695	6,140,974	617,000,000	0	0	-
min_flowiat	-1,247	21	1,076,195	59,947,704	224	405	100.0
max_flowiat	-1	962,728	109,547,639	646,287,731	45	0	100.0
mean_flowiat	0	144,230	19,900,000	64,700,000	0	0	100.0
std_flowiat	0	192,278	33,700,000	204,000,000	0	0	-
min_active	-1	5,949,131	120,988,572	646,565,983	28,663	0	100.0
mean_active	0	0	110,000,000	647,000,000	0	0	100.0
max_active	-1	10,316,202	122,147,146	646,565,983	28,663	0	100.0
std_active	0	0	23,200,000	184,000,000	0	0	-
min_idle	-1	4,993,314	120,661,941	646,287,731	28,663	0	100.0
mean_idle	0	0	109,000,000	646,000,000	0	0	100.0
max_idle	-1	10,000,610	121,615,880	646,287,731	28,663	0	100.0
std_idle	0	0	23,100,000	185,000,000	0	0	-

This also explains the earlier puzzle that A1 and A2 rows never match. As published (names ignored), 0 A1 rows equal an A2 row in all 23 values at any timeout, and only 2,364 of 18,758 at 15 s (1,501, 2,190 and 494 at 30, 60 and 120 s) contain the same set of values in another order. Two of the 23 values mean different things in the two families, and 9 of the 12 A1/B headers shift the names.

2.4 File-family decision

Because A1/B and A2 hold the same flows with two different feature pairs, we had three options: use each task's own files, rebuild A1 and B from the A2 files, or merge both into 25 features. We use **each task's own files**: A1 and B with the paper's 23 features, A2 with its 23. These are the files the paper trained on for each scenario, so our comparison with it stays matched. A rebuild would have replaced two of the paper's features, and a merge is ambiguous wherever a key repeats. For the two-step classifier in E10, both steps use the A1/B layout: the 7-class step is trained on B rows split by VPN (the same flows as A2).

One column needed repair. In A1 120 s, `flowPktsPerSecond` disagrees with A2 on 99.97% of matched rows, while B 120 s, identical to A1 120 s in the other 22 columns of every row, agrees with A2 on 100%. We restore the column from B 120 s, which changes 10,288 of 10,782 rows.

2.5 Cleaning rules

Every rule is in `configs/base.yaml` and is applied by `scripts/prepare_data.py` in this order. Each file gets one line per rule in the cleaning log, including rules that change nothing.

#	Step	Rule	Reason
1	parse	Strip trailing commas from header lines; skip comma-only data lines	Export glitch; a comma-only line (one in each 60 s file of A1 and B) is not a flow
2	column map	Rename with `COLUMN_MAP`	Section 2.3
3	restore	A1 120 s `flowPktsPerSecond` from B 120 s	Section 2.4
4-5	missing, infinite	Count; none found	No imputation needed
6-9	-1 codes	Keep	-1 means "not measurable": fewer than 2 packets in a direction, a single-packet flow, or no active/idle period. The matching mean or total is 0 on the same rows
10	single-packet flows	Keep `duration` = 0 flows	Real flows; dropping them would bias the data toward long flows
11	negative `min_flowiat`	Keep (down to -1,247 us)	Out-of-order capture timestamps
12	rounding	Keep means, stds and rates >= 10^7 as stored	Already rounded to 3 significant figures in the source; can't be undone
13	packet size	Keep flows outside 40-1,500 B per packet	Header-only packets (down to 30 B) and segments merged at capture (up to about 7 kB) are real
14	exact duplicates	Drop within each file before splitting, keep the first copy	A copy in both train and test would score the model on rows it trained on
15	conflicting duplicates	Keep (one row per label)	Real ambiguity; removing it would make the test set easier than reality
16-19	not done	No normalization, no feature selection, no outlier removal, no class balancing	Scaling and resampling run inside the model pipeline on training folds only; all 23 features are kept

Rows each kept-as-is rule applies to:

rule	A1	A2-VPN	A2-NonVPN	B
fiat_not_measurable	8,846	5,120	3,726	8,846
biat_not_measurable	13,857	8,056	5,801	13,857
flowiat_not_measurable	2,108	1,205	903	2,108
active_idle_not_measurable	32,228	15,753	16,475	32,228
single_packet	2,110	1,207	903	2,110
negative_flowiat	586	300	286	586
rounding	15,217	7,979	7,234	15,217
packet_size	1,380	397	983	1,380

2.6 Dataset summary

file	initial_rows	comma_only_lines_skipped	exact_duplicates_removed	conflicting_rows_kept	conflicting_groups	final_rows	missing_values	features	classes	largest_to_smallest_class	train	val	test
A1 15 s	18,758	0	720	46	23	18,038	0	23	2	1.08:1	12,626	2,706	2,706
A1 30 s	14,651	0	1,041	44	22	13,610	0	23	2	1.03:1	9,526	2,042	2,042
A1 60 s	15,516	1	838	94	47	14,677	0	23	2	1.26:1	10,273	2,202	2,202
A1 120 s	10,782	0	1,258	62	31	9,524	0	23	2	1.08:1	6,666	1,429	1,429
A2-VPN 15 s	9,793	0	413	53	24	9,380	0	23	7	5.79:1	6,566	1,407	1,407
A2-VPN 30 s	7,734	0	790	39	17	6,944	0	23	7	8.39:1	4,860	1,042	1,042
A2-VPN 60 s	6,935	0	412	61	28	6,523	0	23	7	12.50:1	4,565	979	979
A2-VPN 120 s	5,631	0	983	111	52	4,648	0	23	7	15.48:1	3,252	698	698
A2-NonVPN 15 s	8,965	0	271	10	3	8,694	0	23	7	11.85:1	6,085	1,304	1,305
A2-NonVPN 30 s	6,917	0	223	9	3	6,694	0	23	7	32.47:1	4,685	1,004	1,005
A2-NonVPN 60 s	8,580	0	374	34	15	8,206	0	23	7	11.31:1	5,744	1,231	1,231
A2-NonVPN 120 s	5,151	0	210	9	3	4,941	0	23	7	14.37:1	3,458	741	742
B 15 s	18,758	0	684	100	41	18,074	0	23	14	11.85:1	12,651	2,711	2,712
B 30 s	14,651	0	1,013	83	33	13,638	0	23	14	32.47:1	9,546	2,046	2,046
B 60 s	15,516	1	786	174	75	14,729	0	23	14	12.69:1	10,309	2,210	2,210
B 120 s	10,782	0	1,193	173	77	9,589	0	23	14	17.24:1	6,711	1,439	1,439

2.7 Data funnel (data v1)

Only two steps remove rows: the parser (two comma-only lines in total) and exact de-duplication (11,209 rows). Over the 16 core files, 179,120 raw data lines become 167,909 cleaned flows; no value is changed except the restored A1 120 s column.

file	raw data lines	parsed	after duplicates	final	train	val	test
A1 15 s	18,758	18,758	18,038	18,038	12,626	2,706	2,706
A1 30 s	14,651	14,651	13,610	13,610	9,526	2,042	2,042
A1 60 s	15,516	15,515	14,677	14,677	10,273	2,202	2,202
A1 120 s	10,782	10,782	9,524	9,524	6,666	1,429	1,429
A2-VPN 15 s	9,793	9,793	9,380	9,380	6,566	1,407	1,407
A2-VPN 30 s	7,734	7,734	6,944	6,944	4,860	1,042	1,042
A2-VPN 60 s	6,935	6,935	6,523	6,523	4,565	979	979
A2-VPN 120 s	5,631	5,631	4,648	4,648	3,252	698	698
A2-NonVPN 15 s	8,965	8,965	8,694	8,694	6,085	1,304	1,305
A2-NonVPN 30 s	6,917	6,917	6,694	6,694	4,685	1,004	1,005
A2-NonVPN 60 s	8,580	8,580	8,206	8,206	5,744	1,231	1,231
A2-NonVPN 120 s	5,151	5,151	4,941	4,941	3,458	741	742
B 15 s	18,758	18,758	18,074	18,074	12,651	2,711	2,712
B 30 s	14,651	14,651	13,638	13,638	9,546	2,046	2,046
B 60 s	15,516	15,515	14,729	14,729	10,309	2,210	2,210

file	raw data lines	parsed	after duplicates	final	train	val	test
B 120 s	10,782	10,782	9,589	9,589	6,711	1,439	1,439
All 16 files	179,120	179,118	167,909	167,909	117,523	25,191	25,195

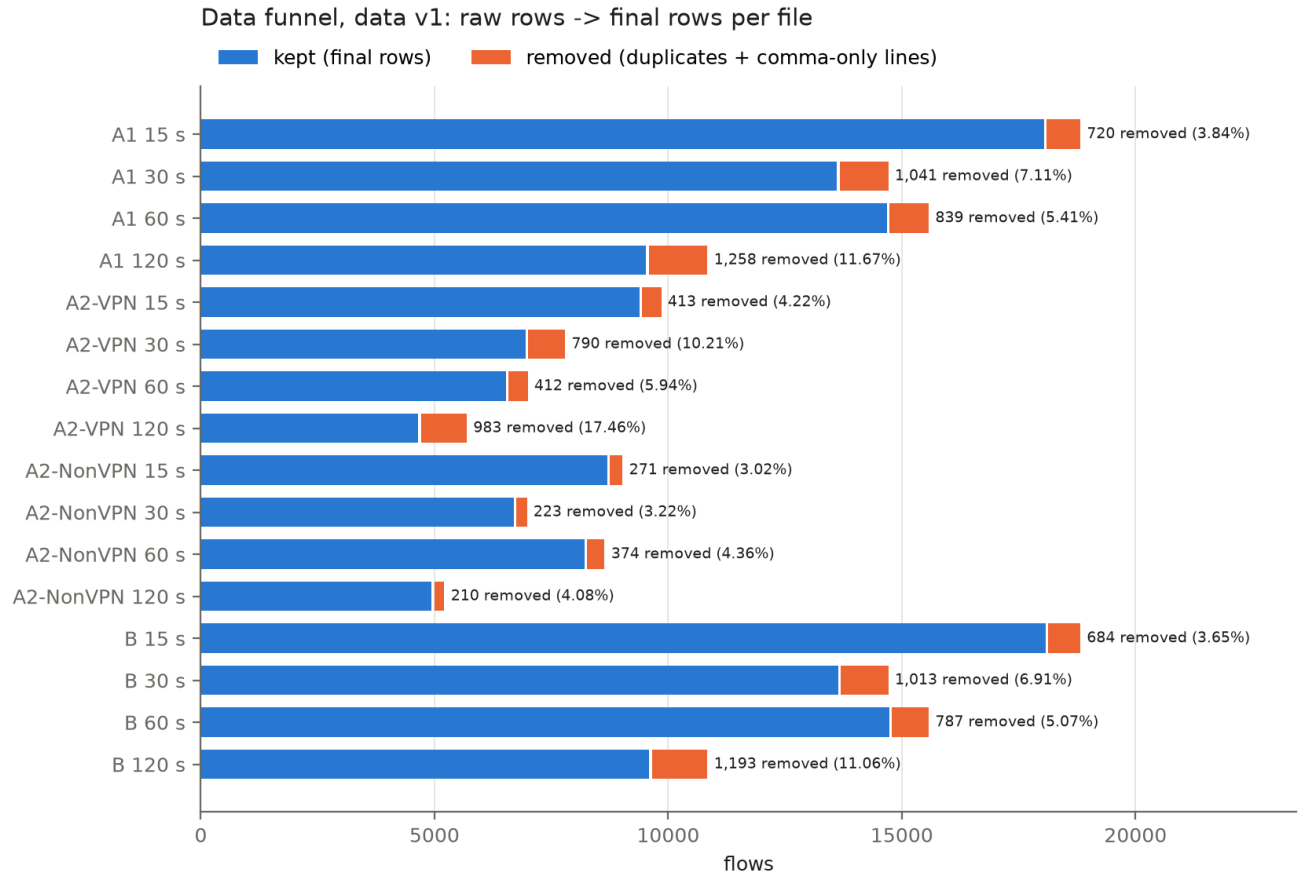


Figure 1. Data funnel

Removals are uneven by class, so they change the class balance.

A1

class	15 s	30 s	60 s	120 s
Non-VPN	278 of 8,965	229 of 6,917	393 of 8,580	216 of 5,151
VPN	442 of 9,793	812 of 7,734	445 of 6,935	1,042 of 5,631

A2-VPN

class	15 s	30 s	60 s	120 s
VPN-BROWSING	0 of 2,500	0 of 2,500	37 of 2,500	256 of 2,500
VPN-CHAT	38 of 1,196	38 of 780	36 of 514	39 of 349
VPN-STREAMING	0 of 475	0 of 298	0 of 197	0 of 145
VPN-MAIL	59 of 491	483 of 848	59 of 383	420 of 722
VPN-VOIP	5 of 2,271	5 of 1,299	9 of 1,620	5 of 386
VPN-P2P	262 of 928	230 of 851	218 of 823	215 of 813
VPN-FT	49 of 1,932	34 of 1,158	53 of 898	48 of 716

A2-NonVPN

class	15 s	30 s	60 s	120 s
BROWSING	0 of 2,500	0 of 2,500	0 of 2,500	0 of 2,500
CHAT	3 of 890	7 of 595	24 of 778	3 of 242
STREAMING	32 of 482	31 of 342	31 of 252	31 of 208
MAIL	11 of 249	4 of 81	117 of 849	11 of 185
VOIP	5 of 2,826	3 of 1,438	36 of 1,829	3 of 392
P2P	59 of 1,000	7 of 1,000	4 of 1,000	2 of 1,000
FT	161 of 1,018	171 of 961	162 of 1,372	160 of 624

B

class	15 s	30 s	60 s	120 s
BROWSING	0 of 2,500	0 of 2,500	0 of 2,500	0 of 2,500
CHAT	3 of 890	7 of 595	24 of 778	3 of 242
STREAMING	32 of 482	31 of 342	31 of 252	31 of 208
MAIL	11 of 249	4 of 81	117 of 849	11 of 185
VOIP	5 of 2,826	3 of 1,438	36 of 1,829	3 of 392
P2P	59 of 1,000	7 of 1,000	4 of 1,000	2 of 1,000
FT	161 of 1,018	171 of 961	162 of 1,372	160 of 624
VPN-BROWSING	0 of 2,500	0 of 2,500	37 of 2,500	256 of 2,500
VPN-CHAT	38 of 1,196	38 of 780	36 of 514	39 of 349
VPN-STREAMING	0 of 475	0 of 298	0 of 197	0 of 145
VPN-MAIL	59 of 491	483 of 848	59 of 383	420 of 722
VPN-VOIP	5 of 2,271	5 of 1,299	9 of 1,620	5 of 386
VPN-P2P	262 of 928	230 of 851	218 of 823	215 of 813
VPN-FT	49 of 1,932	34 of 1,158	53 of 898	48 of 716

file	groups	rows	test rows with a conflicting twin in train
A1 15 s	23	46	4
A1 30 s	22	44	8
A1 60 s	47	94	9
A1 120 s	31	62	7
A2-VPN 15 s	24	53	9
A2-VPN 30 s	17	39	4
A2-VPN 60 s	28	61	6
A2-VPN 120 s	52	111	11
A2-NonVPN 15 s	3	10	3
A2-NonVPN 30 s	3	9	2
A2-NonVPN 60 s	15	34	6
A2-NonVPN 120 s	3	9	1
B 15 s	41	100	15
B 30 s	33	83	15
B 60 s	75	174	25
B 120 s	77	173	21

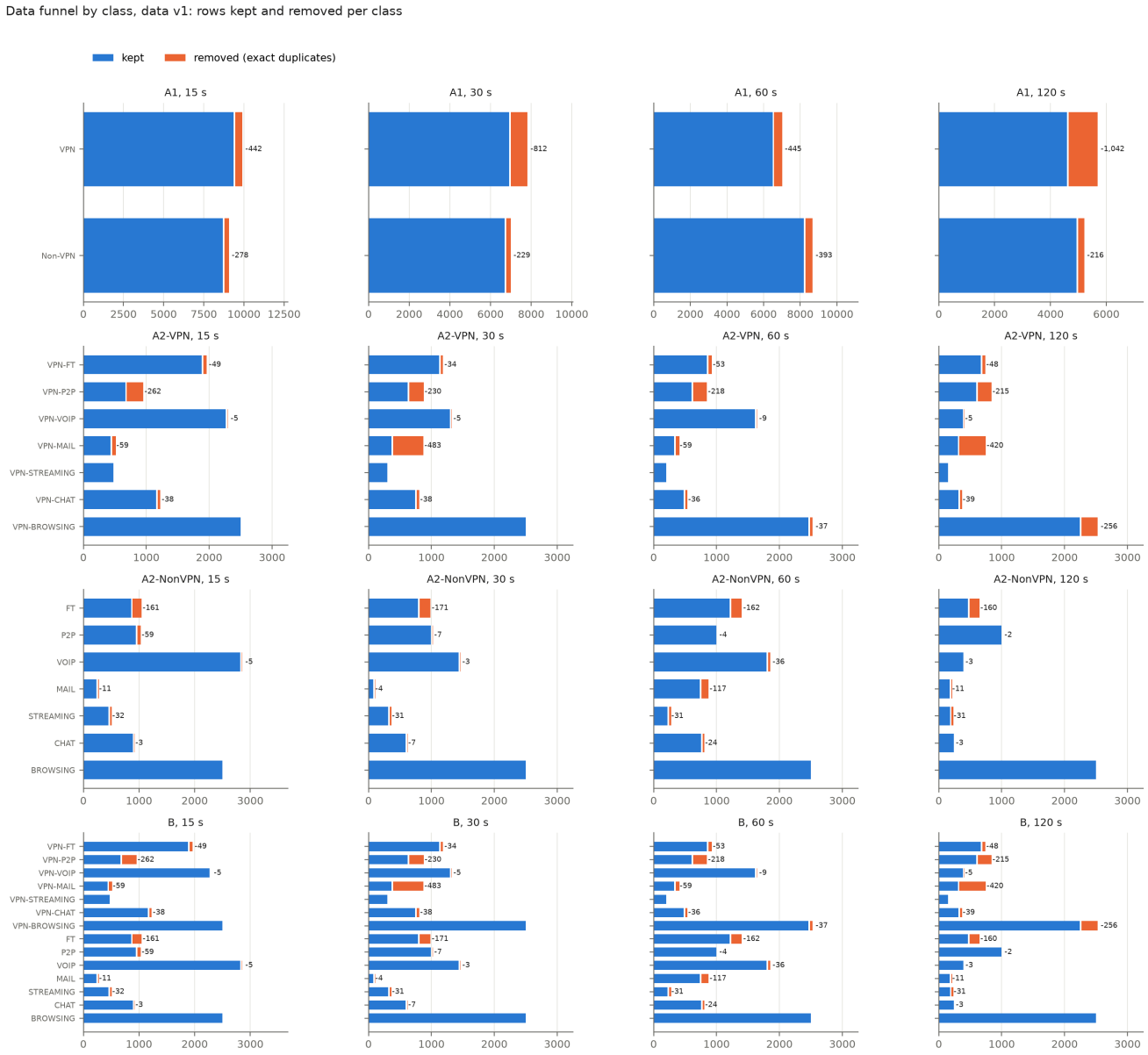


Figure 2. Data funnel by class

Per class, from raw rows to the final train / validation / test sets:

A1

class	timeout	raw	final	train	val	test
Non-VPN	15 s	8,965	8,687	6,081	1,303	1,303
VPN	15 s	9,793	9,351	6,545	1,403	1,403
Non-VPN	30 s	6,917	6,688	4,681	1,004	1,003
VPN	30 s	7,734	6,922	4,845	1,038	1,039
Non-VPN	60 s	8,580	8,187	5,731	1,228	1,228
VPN	60 s	6,935	6,490	4,542	974	974
Non-VPN	120 s	5,151	4,935	3,454	741	740
VPN	120 s	5,631	4,589	3,212	688	689

A2-VPN

class	timeout	raw	final	train	val	test
VPN-BROWSING	15 s	2,500	2,500	1,750	375	375

class	timeout	raw	final	train	val	test
VPN-CHAT	15 s	1,196	1,158	810	174	174
VPN-STREAMING	15 s	475	475	333	71	71
VPN-MAIL	15 s	491	432	302	65	65
VPN-VOIP	15 s	2,271	2,266	1,586	340	340
VPN-P2P	15 s	928	666	466	100	100
VPN-FT	15 s	1,932	1,883	1,319	282	282
VPN-BROWSING	30 s	2,500	2,500	1,750	375	375
VPN-CHAT	30 s	780	742	520	111	111
VPN-STREAMING	30 s	298	298	208	45	45
VPN-MAIL	30 s	848	365	255	55	55
VPN-VOIP	30 s	1,299	1,294	906	194	194
VPN-P2P	30 s	851	621	435	93	93
VPN-FT	30 s	1,158	1,124	786	169	169
VPN-BROWSING	60 s	2,500	2,463	1,724	369	370
VPN-CHAT	60 s	514	478	334	72	72
VPN-STREAMING	60 s	197	197	139	29	29
VPN-MAIL	60 s	383	324	227	49	48
VPN-VOIP	60 s	1,620	1,611	1,127	242	242
VPN-P2P	60 s	823	605	423	91	91
VPN-FT	60 s	898	845	591	127	127
VPN-BROWSING	120 s	2,500	2,244	1,570	337	337
VPN-CHAT	120 s	349	310	216	47	47
VPN-STREAMING	120 s	145	145	101	22	22
VPN-MAIL	120 s	722	302	212	45	45
VPN-VOIP	120 s	386	381	267	57	57
VPN-P2P	120 s	813	598	418	90	90
VPN-FT	120 s	716	668	468	100	100

A2-NonVPN

class	timeout	raw	final	train	val	test
BROWSING	15 s	2,500	2,500	1,750	375	375
CHAT	15 s	890	887	621	133	133
STREAMING	15 s	482	450	315	67	68
MAIL	15 s	249	238	166	36	36
VOIP	15 s	2,826	2,821	1,975	423	423
P2P	15 s	1,000	941	659	141	141
FT	15 s	1,018	857	599	129	129
BROWSING	30 s	2,500	2,500	1,750	375	375
CHAT	30 s	595	588	412	88	88
STREAMING	30 s	342	311	217	47	47
MAIL	30 s	81	77	53	12	12

class	timeout	raw	final	train	val	test
VOIP	30 s	1,438	1,435	1,005	215	215
P2P	30 s	1,000	993	695	149	149
FT	30 s	961	790	553	118	119
BROWSING	60 s	2,500	2,500	1,750	375	375
CHAT	60 s	778	754	528	113	113
STREAMING	60 s	252	221	155	33	33
MAIL	60 s	849	732	512	110	110
VOIP	60 s	1,829	1,793	1,255	269	269
P2P	60 s	1,000	996	697	150	149
FT	60 s	1,372	1,210	847	181	182
BROWSING	120 s	2,500	2,500	1,750	375	375
CHAT	120 s	242	239	167	36	36
STREAMING	120 s	208	177	124	26	27
MAIL	120 s	185	174	122	26	26
VOIP	120 s	392	389	273	58	58
P2P	120 s	1,000	998	698	150	150
FT	120 s	624	464	324	70	70

B

class	timeout	raw	final	train	val	test
BROWSING	15 s	2,500	2,500	1,750	375	375
CHAT	15 s	890	887	621	133	133
STREAMING	15 s	482	450	315	68	67
MAIL	15 s	249	238	166	36	36
VOIP	15 s	2,826	2,821	1,975	423	423
P2P	15 s	1,000	941	659	141	141
FT	15 s	1,018	857	600	128	129
VPN-BROWSING	15 s	2,500	2,500	1,750	375	375
VPN-CHAT	15 s	1,196	1,158	810	174	174
VPN-STREAMING	15 s	475	475	333	71	71
VPN-MAIL	15 s	491	432	302	65	65
VPN-VOIP	15 s	2,271	2,266	1,586	340	340
VPN-P2P	15 s	928	666	466	100	100
VPN-FT	15 s	1,932	1,883	1,318	282	283
BROWSING	30 s	2,500	2,500	1,750	375	375
CHAT	30 s	595	588	412	88	88
STREAMING	30 s	342	311	217	47	47
MAIL	30 s	81	77	54	11	12
VOIP	30 s	1,438	1,435	1,005	215	215
P2P	30 s	1,000	993	695	149	149
FT	30 s	961	790	553	119	118

class	timeout	raw	final	train	val	test
VPN-BROWSING	30 s	2,500	2,500	1,750	375	375
VPN-CHAT	30 s	780	742	520	111	111
VPN-STREAMING	30 s	298	298	208	45	45
VPN-MAIL	30 s	848	365	255	55	55
VPN-VOIP	30 s	1,299	1,294	906	194	194
VPN-P2P	30 s	851	621	435	93	93
VPN-FT	30 s	1,158	1,124	786	169	169
BROWSING	60 s	2,500	2,500	1,750	375	375
CHAT	60 s	778	754	528	113	113
STREAMING	60 s	252	221	155	33	33
MAIL	60 s	849	732	512	110	110
VOIP	60 s	1,829	1,793	1,255	269	269
P2P	60 s	1,000	996	698	149	149
FT	60 s	1,372	1,210	847	182	181
VPN-BROWSING	60 s	2,500	2,463	1,724	369	370
VPN-CHAT	60 s	514	478	334	72	72
VPN-STREAMING	60 s	197	197	138	30	29
VPN-MAIL	60 s	383	324	227	48	49
VPN-VOIP	60 s	1,620	1,611	1,127	242	242
VPN-P2P	60 s	823	605	423	91	91
VPN-FT	60 s	898	845	591	127	127
BROWSING	120 s	2,500	2,500	1,750	375	375
CHAT	120 s	242	239	167	36	36
STREAMING	120 s	208	177	124	26	27
MAIL	120 s	185	174	122	26	26
VOIP	120 s	392	389	273	58	58
P2P	120 s	1,000	998	698	150	150
FT	120 s	624	464	324	70	70
VPN-BROWSING	120 s	2,500	2,244	1,570	337	337
VPN-CHAT	120 s	349	310	217	47	46
VPN-STREAMING	120 s	145	145	101	22	22
VPN-MAIL	120 s	722	302	212	45	45
VPN-VOIP	120 s	386	381	267	57	57
VPN-P2P	120 s	813	598	418	90	90
VPN-FT	120 s	716	668	468	100	100

In sentences. For task A1 at 15 s, 18,758 flows were parsed. After removing 720 exact duplicates (442 VPN, 278 non-VPN), 18,038 unique flows remained. All 23 features were kept without feature selection or normalization, and the flows were split 12,626 / 2,706 / 2,706 into training, validation and test sets. Duplicates are most common at the 120 s timeout: 1,258 of 10,782 A1 flows (11.67%) and 983 of 5,631 A2-VPN flows (17.46%). They hit some classes much harder than others: VPN-MAIL lost 483 of 848 flows at 30 s and 420 of 722 at 120 s, FT lost 160-171 flows at every timeout, and BROWSING lost none. A1 loses slightly more rows than B at the same timeout (720 vs 684 at 15 s) because flows that share features but differ in traffic type

become exact duplicates once both are labelled with the same VPN / non-VPN label. After cleaning, no exact duplicate crosses train and test in any file; 1-25 test flows per file have a twin in training with the same features but a different label.

Deliberately not done, and why. No values were imputed (none are missing). No -1 code was replaced and nothing was set to zero, because -1 has a meaning. No flag columns were added, since the matching zero mean already marks the same rows and flags would change the paper's 23-feature set. No rows were dropped as outliers, because long, bursty or single-packet flows are real traffic. No features were scaled or selected and no classes were rebalanced at data level; all of that happens inside model pipelines, fitted on training data only (Section 3).

2.8 Timeouts

Within each task the feature names, their order and their units are the same at all four timeouts. The number of flows falls as the timeout grows (A1: 18,038 at 15 s to 9,524 at 120 s), and the class mix moves a lot. One class's share changes by up to 24.6 percentage points in A2-NonVPN (VOIP 32.5% at 15 s vs 7.9% at 120 s; BROWSING 28.8% vs 50.6%), 21.6 in A2-VPN, 12.2 in B and 7.6 in A1. A difference in scores between timeouts is therefore confounded by class mix, and we do not attribute it to the timeout alone.



Figure 3. Class mix per timeout

The 60 s files differ in what some classes look like. At 15, 30 and 120 s, non-VPN VOIP flows send about 100 packets per second with a 20 ms median forward gap, and VPN-VOIP flows about 2 with a gap of about 960 ms. At 60 s the two are almost reversed (2.1 packets/s and 962 ms for VOIP, 16.5 packets/s and 18 ms for VPN-VOIP), and non-VPN FT drops from 1,237-3,739 to 56.5 packets/s. It is not a label swap. Of 7,041 flows found identically in more than one timeout file, 210 change label between timeouts; they are mostly 2-packet flows, and no VoIP pair is among them. The 60 s files are also the only ones exported with the paper-style header, so they may come from a different export run. The ARFF files can't settle the cause. Results at 60 s are therefore read with care and the issue is listed under Limitations (Section 7).

class, timeout	flows	median packets per second	median mean_fiat (us)	% mean_fiat 15-25 ms	% no active/idle period
VOIP 15s	2,821	100.1	19994.9	78.1	99.4
VOIP 30s	1,435	100.0	19998.1	77.1	98.9
VOIP 60s	1,793	2.1	961624.6	31.2	51.5
VOIP 120s	389	100.0	19999.8	73.5	95.4
FT 15s	857	1236.7	1042.8	1.2	69.4
FT 30s	790	2155.9	768.0	1.5	72.0
FT 60s	1,210	56.5	34444.3	7.8	80.1

class, timeout	flows	median packets per second	median mean_fiat (us)	% mean_fiat 15-25 ms	% no active/idle period
FT 120s	464	3738.5	507.5	0.4	78.9
VPN-VOIP 15s	2,266	2.2	960175.4	42.9	64.4
VPN-VOIP 30s	1,294	2.1	960284.3	37.7	56.6
VPN-VOIP 60s	1,611	16.5	18295.3	15.7	71.6
VPN-VOIP 120s	381	2.1	960800.7	32.3	49.3
VPN-FT 15s	1,883	7.1	192693.4	0.7	55.1
VPN-FT 30s	1,124	4.0	416771.2	0.8	35.0
VPN-FT 60s	845	6.1	286898.0	1.4	45.1
VPN-FT 120s	668	7.5	187957.4	2.1	48.8

2.9 The Kaggle copy used by Isnan et al.

The Kaggle file used by Isnan et al. [13] has 59,706 rows and the A2 header. Compared as a multiset of rows, it equals the eight A2 files (VPN and non-VPN at all four timeouts) stacked: no row is left over at 12 significant digits, and the labels are the same. It has 40,987 unique rows, their count after removing 18,719 duplicates. Only 3,676 of those duplicates lie within one file; 7,093 distinct flows appear in two (2,475), three (1,286) or four (3,332) timeout files. Most of their "duplicates" are therefore the same flow built with different timeouts, and a random 80/20 split of the stacked file mixes all four timeout settings in training and test. The file also carries the A2 feature pair (`total_*`), not the paper's (`std_*`). E16 reproduces their setup and measures the effect of keeping timeouts apart (Section 4): the drop it finds is explained by training-set size, not by the mixing itself.

2.10 Data versions

Version	Date	Author	What changed	Files	Rows (raw lines -> final)	Commit
v1	2026-10-27	utsav-s-r	First cleaned version	16 core files (A1, A2-VPN, A2-NonVPN, B x 15/30/60/120 s)	179,120 -> 167,909	0a36896

Figures from the exploration notebook (`notebooks/01_eda.ipynb`): class counts per task and timeout (`class_counts_v1.png`), feature distributions by class at every timeout (`feature_boxes_<task>_<timeout>s.png`, 16 figures) and feature correlations (`correlation_<task>_15s.png`, all four tasks), the data funnel by class (`data_funnel_by_class_v1.png`). The notebook records two findings that matter later. Classes differ by orders of magnitude on rate and gap features (VPN-STREAMING: median 202 packets/s; VPN-VOIP: 2.2). Many features are near-duplicates: 19 of 253 pairs in A1 and B (27 in A2-VPN, 20 in A2-NonVPN) have Spearman |rho| >= 0.9, mostly within the active/idle group. Feature importances will therefore be shared out among correlated features.

3 Method

3.1 Pipeline overview

All code is in the project repository (Python 3.12). Dependencies are managed with uv; exact package versions are fixed in `uv.lock`, so every team member and every rerun uses identical library versions. The main libraries are pandas and NumPy (data), liac-arff and pyarrow (reading ARFF, writing Parquet), scikit-learn [3] (models, pipelines, metrics), MLflow [8] (run tracking), matplotlib and seaborn (figures) and SHAP [6] (explanations). XGBoost [4], LightGBM [5], Optuna [7] and imbalanced-learn [9] were added together in one commit before the first Phase 3 run, so every Phase 3 run uses the same locked library versions.

The flow of data and results is:

1. **Raw data.** The 20 ARFF files of ISCXVPN2016 [2] are kept unchanged; their SHA-256 hashes are recorded in `data/arff_sha256.txt` and checked on every machine.
2. **Preparation.** `scripts/prepare_data.py` parses the files, applies the verified column-name map and the cleaning rules, and writes one Parquet file per task and timeout to a versioned folder `data/processed/vN/`. [Column-name check and cleaning rules: E00.]
3. **Training.** `scripts/run_experiment.py` reads one YAML configuration file, builds a scikit-learn Pipeline (preprocessing + model), runs 5-fold cross-validation on the training set, fits on the training set and scores the validation set.
4. **Tracking.** Every run is logged to MLflow (Section 3.6).
5. **Reporting.** `scripts/export_history.py` exports every run to `reports/history.csv`; `scripts/results_tables.py` builds every result table from it, and the notebooks (written for Phase 3 by `scripts/make_notebooks.py`) draw the figures from it, so no number in this report is copied by hand.

3.2 Data versions

Any change that alters the cleaned data (a cleaning rule, the column map, the parser, row filtering or the split seed) creates a new data version (v1, v2, …) in its own folder, which is never overwritten. Each version has a manifest (file hashes, row and label counts, column map, cleaning rules) and a step-by-step cleaning log listing, for every step, the rows before and after, rows removed or changed (overall and per class), the rule and its reason. Steps that change nothing and decisions not to do something are logged too. Every run records the data version and its hash. After a version change, the current best model is rerun as its own experiment so the effect of the change is measured. [Data versions: see Section 2 and the data changelog.]

3.3 Splits and evaluation protocol

For each task and timeout the cleaned data is split once into 70% training, 15% validation and 15% test sets, stratified by class, with random seed 42. The row indices are saved inside the data version, so every run on that version uses exactly the same rows.

- Hyperparameters are chosen by 5-fold stratified cross-validation inside the training set only.
- Candidate models are compared on the validation set.
- The chosen configuration is retrained on training + validation data and scored **once** on the test set, at the final milestone, with 95% confidence intervals from 1,000 bootstrap resamples. A comparison set (the best validation variant of each method type per task, Section 3.8) is fixed before the test set is opened and scored the same way, once; it never changes the chosen configuration. A difference between two models of a task is tested with a paired bootstrap on the same test rows, Holm-corrected for the number of comparisons.
- Scaling, transforms and resampling are steps inside the pipeline, so they are fitted on training folds only and never see validation or test rows.

- Every random step (split, models, tuning sampler, oversampling, bootstrap) uses seed 42. Only the learning-curve and seed-stability experiments (E13, E14) use seeds 42–51, on purpose.

Paper benchmark. To compare with Draper-Gil et al. [1], the decision tree and KNN are also run with the paper's own protocol: 10-fold cross-validation on the full file with Weka's default settings (KNN with k = 1; C4.5 with pruning confidence 0.25 and at least 2 samples per leaf). These results are reported in their own table. scikit-learn's decision tree with the entropy criterion is a close substitute for C4.5 [11], not an exact copy, and is called a C4.5 approximation.

Session-aware check. Flows from one recording session may fall on both sides of a random split and inflate scores. A group-aware split would keep each session on one side, but the ARFF files hold only the 23 timing features and the label: no capture-file name, address, port or timestamp identifies a session (E00). E11 therefore cannot run, and this is listed as a limitation (Section 7).

3.4 Models

Model	Settings	Experiment
Majority class	Always predicts the most frequent class (the floor)	E01
Decision tree	Entropy criterion (C4.5 approximation); then tuned	E02, E17
KNN	k = 5, standard scaling; signed log + standard or robust scaling; then tuned	E03, E04, E07
Logistic Regression	Standard scaling; then tuned, signed log optional	E03, E17
SVM, RBF kernel	Standard scaling; then tuned, signed log optional	E03, E17
Random Forest	300 trees, default settings; then tuned	E05, E07, E17
XGBoost, LightGBM	Default settings; then tuned	E06, E07, E17
Class balancing	Class weights, random oversampling, SMOTE (inside the pipeline)	E08
Two-step model (task B)	VPN / non-VPN model, then one 7-class model per group	E10

Every baseline is run on all tasks (A1; A2-VPN and A2-NonVPN as separate models; B) at all four timeouts. Improvement experiments change one thing at a time against the current best model: preprocessing for KNN (E04), tuning with Optuna, 50 trials per model with the same budget for the best baseline (E07), class imbalance handling (E08: none, class weights, random oversampling, SMOTE [10]), feature-group ablation (E09), direct vs two-step classification for task B (E10), and the same 50-trial tuning budget for every remaining method type (E17). The chosen models are then described by a learning curve (E13) and a split-seed stability check (E14), and a published result on a copy of this dataset is reproduced and corrected (E16). Each experiment states its hypothesis and a numeric pass/fail threshold before it is run.

3.5 Metrics

The main metric is **macro F1**, which weights every class equally, so small classes count as much as large ones. We also report per-class precision, recall and F1; weighted F1; accuracy; balanced accuracy; confusion matrices and the five most frequent confusion pairs; ROC-AUC (task A1); cross-validation mean and standard deviation; the gap between cross-validation and held-out scores; training time, prediction time per 1,000 flows, model size and peak memory; and, for the final models, Expected Calibration Error with a reliability diagram (E15). Comparisons with the paper use precision against precision and recall against recall.

3.6 Tracking and reproducibility

Every run is logged automatically to MLflow with: the experiment ID, the person and machine that ran it (operating system, CPU, memory), the git commit, whether there were uncommitted changes, the hash of the dependency lock file, the configuration file, the data version and hash, all settings, all metrics, timing, model size and memory, plots, and the trained model file (every official run except individual tuning trials). Official runs are made on one machine, so timings are comparable, and only from code that is committed and pushed to the shared repository; the run script refuses otherwise. Each experiment also has a written entry

(hypothesis, threshold, change, result, verdict, decision), and milestones are marked with git tags (v0.1-baseline, v0.2-tuned, v1.0-final).

3.7 Preprocessing for distance-based models: the signed log

KNN, logistic regression and the RBF-kernel SVM depend on distances or linear scales, and the raw timing features span six or more orders of magnitude (microseconds up to about 1.2e8), so a few very large values dominate. A log compresses them, but plain log1p is not usable here: the features hold -1 codes meaning "not measurable" (log1p(-1) is minus infinity) and 586 negative minimum inter-arrival times (E00). We therefore apply the **signed log** sign(x)·log1p(|x|) as a step inside the pipeline, before scaling. It is defined for every value, keeps the order of values, keeps -1 apart from 0, and is a fixed formula: nothing is learned from data, so it cannot leak information from validation or test rows. Data v1 is not changed. Alternatives not chosen: shifting each column so its minimum is 0 before the log (a shift learned from the training data, different per column, and -1 merges into small values); logging only the non-negative columns (inconsistent, and large values stay uncompressed); replacing -1 by 0 plus indicator columns (changes the feature set, so E04 would change two things at once). E04 compares the signed log with StandardScaler and with RobustScaler; in E17 logistic regression and the SVM choose it inside their search (both chose it in every task). Tree models are not affected by any order-keeping transform, so they get none.

3.8 Phase 3 rules and decisions

These rules were fixed before the Phase 3 runs (28 Oct) unless stated otherwise; each experiment entry repeats the ones it uses before it runs.

- **One change at a time.** Each experiment changes one thing against the **current best** model of each task: the highest 15 s validation macro F1 so far, whatever the margin.
- **Winner rule.** Every experiment runs first at 15 s for all four tasks. A model whose 15 s validation macro F1 beats the task's current best by at least 0.03 is rerun at 30, 60 and 120 s with its 15 s settings (nothing is re-tuned per timeout). Why 0.03: it is above the largest Phase 2 cross-validation standard deviation (0.025), so a smaller gain could be noise from the split; a fixed number set before running keeps the choice honest. Not chosen: rerunning the top two per task, or running everything at every timeout (about four times the runs, and no screen-then-confirm design).
- **Tuning (E07, E17).** Optuna with the TPE sampler (seed 42), 50 trials per search, trials run one after another (parallel trials are not reproducible). Each trial is scored by 5-fold cross-validation macro F1 on the training rows only. E07 tunes, per task, the best two of Random Forest, XGBoost and LightGBM plus KNN with the same budget, so no model wins only because it was tuned more. E17 (added 28 Oct after E10) gives the same budget to every other method type, so the comparison of method types is between tuned models. Search ranges were written into the configurations before running. The best trial is retrained and scored like any other run; trials keep their settings and scores only.
- **Balancing (E08).** Inside the pipeline (imbalanced-learn), so only training folds are resampled. XGBoost has no class-weight option, so its "balanced" step uses per-row sample weights. SMOTE uses 5 neighbours; the smallest training class in any task file has 53 rows (MAIL, A2-NonVPN 30 s), about 42 per cross-validation fold, so it is always possible.
- **Two-step model (E10).** Both steps train on task B's training rows only; the VPN / non-VPN label comes from the B label. The A1 split is never used, because A1 and B hold the same flows split differently, so an A1-trained model would have seen B validation flows.
- **Analysis experiments (E09, E13, E14)** describe the chosen models and change nothing. If a feature set in E09 had scored at least 0.03 above all 23 features, the work would have stopped for a decision, because choosing features on validation scores risks overfitting the choice. E13 and E14 use other seeds on purpose and never touch the seed-42 test rows.
- **Reproducing Isnan et al. (E16)** uses its own data version (kaggle-k1, built by a committed script with its own manifest and hashes); data v1 is untouched, and no E16 result feeds model selection.

- **Cost and resources.** Every model runs on one thread (XGBoost and LightGBM included), with the five cross-validation folds in parallel. This keeps results deterministic and timings comparable with Phase 2. Peak memory is measured twice: with Python's tracemalloc (comparable with Phase 2) and as the peak memory of a fresh process doing the fit, which also sees the native memory of XGBoost and LightGBM. Every model is saved with joblib compression level 3. This is lossless and cuts a 300-tree Random Forest from 171.9 MiB to 22.4 MiB. Both the compressed and the uncompressed size are logged.
- **Final choice (decided 28 Oct after E10, a deviation from the rule set before Phase 3).** The rule fixed before Phase 3 treated any gap below 0.03 as a tie, broken in favour of the simpler model. Applied as written, it would have chosen KNN or the SVM at 0.0146-0.0286 below the best model. The cross-validation standard deviations of the chosen models (0.0028-0.0058) are far below 0.03, and the goal is the best score on unseen traffic. The final configuration per task is therefore the highest validation macro F1 among all model-selection runs (any experiment, any timeout). The only fallback is a saved model file above 50 MB (GitHub's warning size), which none of the chosen models reaches. The old rule's picks are scored on the test set too (E12), so the cost of either rule is visible.
- **Comparison set.** Fixed in `configs/final/compare/` before the test set is opened: per task, the best 15 s validation variant of each of 7 method types (decision tree, KNN, logistic regression, SVM-RBF, Random Forest, XGBoost, LightGBM), 28 models. A type's variants are its default, tuned and balanced runs; KNN includes the E03 and E04 preprocessings; the E10 two-step model counts as XGBoost; the majority baseline is a reference, not a member. Only the four headline models are saved in the repository (`models/final/`); the others stay in the run store.

4 Experiments

Each experiment states a hypothesis and a numeric pass/fail threshold before it runs, changes one thing, and ends with a verdict (supported / not supported). Unless stated otherwise, scores are on the **validation** set of the fixed 70/15/15 split (seed 42); models are chosen with 5-fold cross-validation on the training set; the test set is not used until the final experiment (E12).

4.0 E00: data findings and data version v1

E00 checked the dataset before any model was trained: the corrected column names, the file-family decision, the odd-value and duplicate rules, the session question and the Kaggle finding. It produced data version v1 (167,909 flows in 16 task files). Section 2 reports it in full.

4.1 E01: majority-class baseline

What. A model that always predicts the largest training class (`DummyClassifier`), for every task and timeout (16 runs). **Why.** It is the floor every model must beat, and its scores are known in advance, so it also tests the metrics code: accuracy must equal the largest class's share p of the validation set, and macro F1 must equal 2p / ((1 + p) k) for k classes. **Result.** Supported: all 16 runs match both values to 4 decimals (threshold 0.001). The floor in macro F1 is 0.34-0.36 for A1, 0.06-0.10 for A2 and 0.02-0.03 for B. Accuracy alone would look much better (up to 0.56 for A1 and 0.51 for A2-NonVPN 120 s), which is why macro F1 is the main score.

task	timeout	classes	largest_class	p	val_accuracy	expected_macro_f1	val_macro_f1	ok
A1	15 s	2	VPN	0.5185	0.5185	0.3414	0.3414	yes
A1	30 s	2	VPN	0.5083	0.5083	0.337	0.337	yes
A1	60 s	2	Non-VPN	0.5577	0.5577	0.358	0.358	yes
A1	120 s	2	Non-VPN	0.5185	0.5185	0.3415	0.3415	yes
A2-VPN	15 s	7	VPN-BROWSING	0.2665	0.2665	0.0601	0.0601	yes
A2-VPN	30 s	7	VPN-BROWSING	0.3599	0.3599	0.0756	0.0756	yes
A2-VPN	60 s	7	VPN-BROWSING	0.3769	0.3769	0.0782	0.0782	yes
A2-VPN	120 s	7	VPN-BROWSING	0.4828	0.4828	0.093	0.093	yes
A2-NonVPN	15 s	7	VOIP	0.3244	0.3244	0.07	0.07	yes
A2-NonVPN	30 s	7	BROWSING	0.3735	0.3735	0.0777	0.0777	yes
A2-NonVPN	60 s	7	BROWSING	0.3046	0.3046	0.0667	0.0667	yes
A2-NonVPN	120 s	7	BROWSING	0.5061	0.5061	0.096	0.096	yes
B	15 s	14	VOIP	0.156	0.156	0.0193	0.0193	yes
B	30 s	14	BROWSING	0.1833	0.1833	0.0221	0.0221	yes
B	60 s	14	BROWSING	0.1697	0.1697	0.0207	0.0207	yes
B	120 s	14	BROWSING	0.2606	0.2606	0.0295	0.0295	yes

4.2 E02 and E03: decision tree, KNN, Logistic Regression and SVM-RBF

What. Four baselines, each a scikit-learn pipeline, run on all 16 task/timeout files (64 runs): an unpruned entropy decision tree (E02; the closest scikit-learn model to C4.5, so a "C4.5 approximation"), KNN with k = 5 and standard scaling, Logistic Regression and an RBF-kernel SVM, both with standard scaling (E03). Scaling is a pipeline step, fitted on training rows only. **Why.** The tree and KNN are the paper's two algorithms. Logistic Regression and SVM-RBF match the baselines of Isnan et al. [13]. Together they set the bar for the improvements in Phase 3. **Hypotheses and thresholds (written before running).** E02: the tree beats the floor by at least 0.30 in every config and reaches at least 0.80 on A1 at every timeout. E03: all three beat the floor by at least 0.10; KNN is within 0.10 of the tree in at least 12 of 16 configs; Logistic Regression and SVM-

RBF score below the tree in at least 12 of 16 configs, because the features are heavy-tailed (microsecond times up to about 10^8 next to -1 and 0 codes). **Result.** All supported. The tree is the best baseline in all 16 configs (macro F1 0.706-0.890), 0.499-0.765 above the floor. KNN is second everywhere, 0.039-0.101 below the tree (within 0.10 in 15 of 16). Logistic Regression (0.270-0.598) and SVM-RBF (0.274-0.621) are below the tree in all 16.

Validation macro F1 (E01-E03):

task	timeout	majority	tree	knn	logreg	svm_rbf
A1	15 s	0.341	0.890	0.813	0.585	0.602
A1	30 s	0.337	0.864	0.825	0.598	0.610
A1	60 s	0.358	0.857	0.813	0.541	0.545
A1	120 s	0.341	0.857	0.807	0.576	0.621
A2-VPN	15 s	0.060	0.825	0.774	0.432	0.468
A2-VPN	30 s	0.076	0.828	0.788	0.536	0.542
A2-VPN	60 s	0.078	0.775	0.730	0.339	0.330
A2-VPN	120 s	0.093	0.780	0.727	0.332	0.332
A2-NonVPN	15 s	0.070	0.829	0.773	0.449	0.487
A2-NonVPN	30 s	0.078	0.828	0.783	0.486	0.468
A2-NonVPN	60 s	0.067	0.825	0.750	0.447	0.441
A2-NonVPN	120 s	0.096	0.839	0.769	0.482	0.471
B	15 s	0.019	0.759	0.660	0.307	0.296
B	30 s	0.022	0.748	0.647	0.320	0.311
B	60 s	0.021	0.706	0.617	0.271	0.278
B	120 s	0.030	0.716	0.637	0.270	0.274

5-fold cross-validation on the training set, mean +- standard deviation:

task	timeout	majority	tree	knn	logreg	svm_rbf
A1	15 s	0.341 +- 0.000	0.888 +- 0.005	0.821 +- 0.002	0.600 +- 0.011	0.608 +- 0.005
A1	30 s	0.337 +- 0.000	0.873 +- 0.006	0.814 +- 0.010	0.600 +- 0.012	0.619 +- 0.023
A1	60 s	0.358 +- 0.000	0.852 +- 0.010	0.808 +- 0.009	0.551 +- 0.008	0.552 +- 0.006
A1	120 s	0.341 +- 0.000	0.857 +- 0.009	0.803 +- 0.002	0.569 +- 0.008	0.608 +- 0.011
A2-VPN	15 s	0.060 +- 0.000	0.810 +- 0.007	0.767 +- 0.008	0.410 +- 0.014	0.448 +- 0.024
A2-VPN	30 s	0.076 +- 0.000	0.796 +- 0.009	0.769 +- 0.025	0.495 +- 0.016	0.507 +- 0.021
A2-VPN	60 s	0.078 +- 0.000	0.760 +- 0.016	0.708 +- 0.019	0.334 +- 0.022	0.331 +- 0.021
A2-VPN	120 s	0.093 +- 0.000	0.725 +- 0.023	0.692 +- 0.016	0.343 +- 0.019	0.317 +- 0.021
A2-NonVPN	15 s	0.070 +- 0.000	0.862 +- 0.009	0.792 +- 0.010	0.459 +- 0.013	0.479 +- 0.018
A2-NonVPN	30 s	0.078 +- 0.000	0.817 +- 0.015	0.783 +- 0.017	0.459 +- 0.007	0.426 +- 0.008
A2-NonVPN	60 s	0.067 +- 0.000	0.829 +- 0.016	0.740 +- 0.013	0.454 +- 0.015	0.432 +- 0.022
A2-NonVPN	120 s	0.096 +- 0.000	0.807 +- 0.015	0.731 +- 0.013	0.472 +- 0.016	0.441 +- 0.023
B	15 s	0.019 +- 0.000	0.757 +- 0.013	0.663 +- 0.008	0.299 +- 0.010	0.278 +- 0.008
B	30 s	0.022 +- 0.000	0.733 +- 0.008	0.654 +- 0.016	0.323 +- 0.009	0.292 +- 0.008
B	60 s	0.021 +- 0.000	0.703 +- 0.013	0.607 +- 0.010	0.275 +- 0.002	0.261 +- 0.009
B	120 s	0.030 +- 0.000	0.684 +- 0.020	0.607 +- 0.015	0.250 +- 0.005	0.251 +- 0.017

What the tables show:

- **Cross-validation predicts validation.** For every run, |CV mean - validation| is at most 0.054, and CV standard deviations are at most 0.025. A gap of several points between two models is therefore larger than fold-to-fold noise.
- **Small classes.** Logistic Regression and SVM-RBF score F1 = 0.00 on MAIL / VPN-MAIL in most A2 and B configs. So their macro F1 is far below their weighted F1 (gap up to 0.225); the tree's gap is 0.000-0.069. The SVM keeps 55-88% of the training rows as support vectors, a sign that the RBF kernel separates the standard-scaled features poorly. A log transform or a robust scaler is tested in E04.
- **Easiest and hardest classes.** VoIP is the easiest type (tree F1 0.92-0.99 for VOIP and VPN-VOIP). The weakest are CHAT, VPN-CHAT, STREAMING, MAIL and VPN-MAIL (tree F1 0.42-0.71).
- **Most frequent confusion.** In task B the largest confusion for the tree and KNN, at every timeout, is BROWSING vs VPN-BROWSING in both directions (tree: 29-45 validation flows each way). The model recognises browsing, but not whether it went through the VPN.
- **Timeouts.** B and A2-VPN score lower at 60 s and 120 s. The class mix differs between timeouts (Section 2.8), and the 60 s files hold differently shaped VOIP and FT flows (E00 addendum), so this is not read as an effect of the timeout.
- **Cost.** All baselines train in under 3 s. To predict 1,000 flows, the tree needs 0.30 ms (median), KNN 8.0 ms and the SVM 210 ms. Model files range from 1.4 KB (majority) and 4-5 KB (Logistic Regression) to 4.2 MB (SVM).

Per-class extremes for the tree (E02): macro vs weighted F1, their gap, the generalisation gap, and the best and worst class by F1 and by recall:

task	timeout	macro F1	weighted F1	gap	CV - val	best F1	worst F1	best recall	worst recall
A1	15 s	0.89	0.89	0.0	-0.002	VPN 0.89	Non-VPN 0.89	Non-VPN 0.89	VPN 0.89
A1	30 s	0.864	0.864	0.0	0.009	VPN 0.87	Non-VPN 0.86	VPN 0.87	Non-VPN 0.86
A1	60 s	0.857	0.859	0.002	-0.005	Non-VPN 0.87	VPN 0.84	Non-VPN 0.87	VPN 0.85
A1	120 s	0.857	0.857	0.0	0.0	Non-VPN 0.86	VPN 0.85	Non-VPN 0.86	VPN 0.85
A2-VPN	15 s	0.825	0.849	0.024	-0.014	VPN-VOIP 0.99	VPN-CHAT 0.65	VPN-VOIP 0.98	VPN-CHAT 0.66
A2-VPN	30 s	0.828	0.847	0.02	-0.031	VPN-VOIP 0.98	VPN-CHAT 0.67	VPN-STREAMING 0.98	VPN-CHAT 0.70
A2-VPN	60 s	0.775	0.831	0.056	-0.014	VPN-VOIP 0.92	VPN-CHAT 0.63	VPN-VOIP 0.92	VPN-FT 0.65
A2-VPN	120 s	0.78	0.825	0.045	-0.054	VPN-VOIP 0.98	VPN-CHAT 0.61	VPN-VOIP 0.98	VPN-CHAT 0.57
A2-NonVPN	15 s	0.829	0.891	0.061	0.033	VOIP 0.99	STREAMING 0.71	VOIP 0.99	STREAMING 0.70

task	timeout	macro F1	weighted F1	gap	CV - val	best F1	worst F1	best recall	worst recall
A2-NonVPN	30 s	0.828	0.889	0.061	-0.011	VOIP 0.98	STREAMING 0.68	VOIP 0.98	STREAMING 0.57
A2-NonVPN	60 s	0.825	0.878	0.053	0.004	VOIP 0.97	STREAMING 0.58	VOIP 0.97	STREAMING 0.55
A2-NonVPN	120 s	0.839	0.893	0.054	-0.033	VOIP 0.95	CHAT 0.56	BROWSING 0.95	CHAT 0.56
B	15 s	0.759	0.81	0.051	-0.001	VOIP 0.98	VPN-MAIL 0.55	VOIP 0.99	VPN-MAIL 0.52
B	30 s	0.748	0.8	0.053	-0.014	VPN-VOIP 0.98	MAIL 0.55	VPN-VOIP 0.99	MAIL 0.55
B	60 s	0.706	0.775	0.069	-0.002	VOIP 0.92	VPN-CHAT 0.42	VOIP 0.92	VPN-CHAT 0.42
B	120 s	0.716	0.767	0.051	-0.031	VOIP 0.97	STREAMING 0.47	VOIP 1.00	CHAT 0.42

Top 5 confusion pairs of the tree on the validation set (E02):

task	timeout	top 5 confusion pairs (true -> predicted: flows)
A1	15 s	VPN -> Non-VPN: 158; Non-VPN -> VPN: 139
A1	30 s	Non-VPN -> VPN: 142; VPN -> Non-VPN: 135
A1	60 s	Non-VPN -> VPN: 161; VPN -> Non-VPN: 150
A1	120 s	Non-VPN -> VPN: 104; VPN -> Non-VPN: 100
A2-VPN	15 s	VPN-FT -> VPN-CHAT: 28; VPN-CHAT -> VPN-BROWSING: 26; VPN-BROWSING -> VPN-CHAT: 25; VPN-CHAT -> VPN-FT: 20; VPN-FT -> VPN-BROWSING: 19
A2-VPN	30 s	VPN-BROWSING -> VPN-CHAT: 26; VPN-FT -> VPN-BROWSING: 22; VPN-BROWSING -> VPN-FT: 15; VPN-CHAT -> VPN-BROWSING: 12; VPN-CHAT -> VPN-FT: 12
A2-VPN	60 s	VPN-FT -> VPN-BROWSING: 17; VPN-BROWSING -> VPN-FT: 16; VPN-FT -> VPN-CHAT: 14; VPN-BROWSING -> VPN-VOIP: 13; VPN-CHAT -> VPN-FT: 12
A2-VPN	120 s	VPN-FT -> VPN-BROWSING: 18; VPN-BROWSING -> VPN-FT: 16; VPN-CHAT -> VPN-BROWSING: 11; VPN-FT -> VPN-P2P: 9; VPN-P2P -> VPN-BROWSING: 9
A2-NonVPN	15 s	BROWSING -> CHAT: 18; CHAT -> BROWSING: 15; BROWSING -> STREAMING: 12; FT -> BROWSING: 12; FT -> CHAT: 11
A2-NonVPN	30 s	CHAT -> BROWSING: 12; STREAMING -> BROWSING: 11; BROWSING -> CHAT: 10; BROWSING -> FT: 10; FT -> CHAT: 9
A2-NonVPN	60 s	CHAT -> BROWSING: 20; BROWSING -> CHAT: 19; MAIL -> FT: 8; FT -> BROWSING: 7; BROWSING -> MAIL: 6
A2-NonVPN	120 s	CHAT -> BROWSING: 12; P2P -> BROWSING: 8; FT -> BROWSING: 7; BROWSING -> P2P: 6; BROWSING -> CHAT: 5
B	15 s	VPN-BROWSING -> BROWSING: 41; BROWSING -> VPN-BROWSING: 39; VPN-CHAT -> VPN-FT: 25; VPN-FT -> VPN-CHAT: 24; VPN-BROWSING -> VPN-CHAT: 18
B	30 s	VPN-BROWSING -> BROWSING: 44; BROWSING -> VPN-BROWSING: 36; VPN-BROWSING -> VPN-CHAT: 13; VPN-FT -> VPN-CHAT: 12; VPN-CHAT -> VPN-BROWSING: 11
B	60 s	BROWSING -> VPN-BROWSING: 37; VPN-BROWSING -> BROWSING: 29; VPN-MAIL -> MAIL: 15; VPN-P2P -> P2P: 14; VPN-VOIP -> VOIP: 14
B	120 s	BROWSING -> VPN-BROWSING: 45; VPN-BROWSING -> BROWSING: 32; VPN-BROWSING -> VPN-FT: 17; VPN-FT -> VPN-BROWSING: 12; VPN-BROWSING -> VPN-P2P: 10

Model complexity (E02 tree, E03 SVM):

task	timeout	n_train	tree depth	tree nodes	tree leaves	SVM support vectors
A1	15 s	12,626	27	1,789	895	10,292
A1	30 s	9,526	35	1,601	801	7,976
A1	60 s	10,273	27	1,917	959	7,882

task	timeout	n_train	tree depth	tree nodes	tree leaves	SVM support vectors
A1	120 s	6,666	25	1,243	622	5,501
A2-VPN	15 s	6,566	23	1,407	704	4,497
A2-VPN	30 s	4,860	20	1,067	534	3,108
A2-VPN	60 s	4,565	24	1,249	625	3,265
A2-VPN	120 s	3,252	21	985	493	2,368
A2-NonVPN	15 s	6,085	16	743	372	3,329
A2-NonVPN	30 s	4,685	19	751	376	2,751
A2-NonVPN	60 s	5,744	24	1,001	501	3,803
A2-NonVPN	120 s	3,458	18	591	296	1,907
B	15 s	12,651	30	3,409	1,705	11,076
B	30 s	9,546	23	2,901	1,451	8,368
B	60 s	10,309	26	3,437	1,719	8,796
B	120 s	6,711	22	2,481	1,241	5,813

Cost per model over the 16 configs:

model	train time (s) median	train time (s) max	5-fold CV time (s) median	5-fold CV time (s) max	predict ms / 1,000 flows median	predict ms / 1,000 flows max	model file (KB) median	model file (KB) max	peak fit memory (MB) median	peak fit memory (MB) max
majority	0.004	0.008	0.157	2.854	0.091	0.18	1.414	1.523	0.623	1.182
tree	0.112	0.304	0.293	0.652	0.298	0.531	134.49	592.916	1.257	2.392
knn	0.01	0.021	0.182	0.281	8.031	13.338	1242.967	2374.631	2.999	5.722
logreg	0.084	0.428	0.212	0.603	0.378	0.69	3.829	5.196	3.433	8.144
svm_rbf	0.626	2.913	1.61	3.784	209.808	596.678	1175.091	4288.896	3.552	7.781

Peak fit memory is measured with Python's tracemalloc, which counts NumPy arrays but not memory allocated inside compiled libraries (for example libsvm), so SVM memory is a lower bound. Training time comes from a fit without tracemalloc and peak memory from a separate fit of a clone. The first Phase 2 runs (commits 1817c5a and 36c56a1) timed the fit with tracemalloc on, which slowed it (x1.05 tree, x1.00 SVM, x1.19 Logistic Regression, x2.3 KNN in a recheck); all 144 configs were rerun from the fixed code (commit c450c3e) with identical scores, and these tables use the rerun. The first runs also logged process_max_rss_mb, a high-water mark of the whole batch process rather than a per-run figure; it is not used.

The same tables for KNN, Logistic Regression and SVM-RBF, plus accuracy, balanced accuracy, weighted F1 and A1 ROC-AUC for every run, are in `reports/baseline_tables.md` (Appendix).

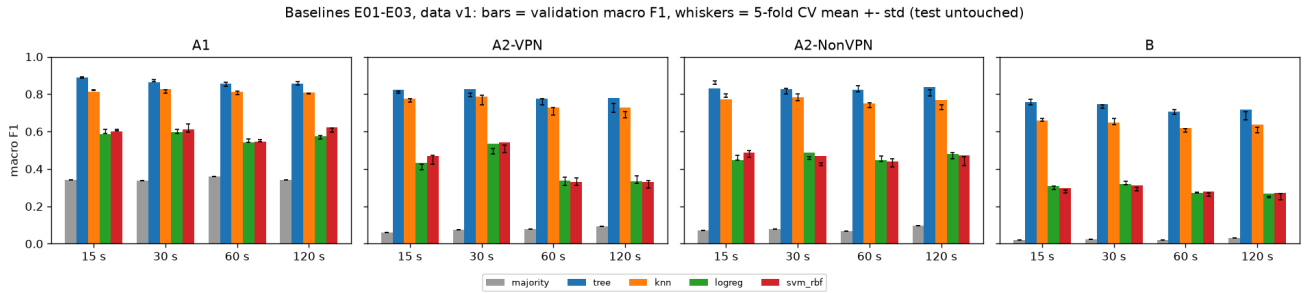


Figure 4. Baselines E01-E03

4.3 Paper benchmark (part of E02 and E03)

What. The paper's own protocol [1]: 10-fold stratified cross-validation on the whole file, with predictions pooled over the folds as Weka reports them, for its two algorithms with Weka's default settings. J48 (C4.5)

defaults are a pruning confidence of 0.25 and at least 2 instances per leaf; scikit-learn has no C4.5 pruning, so our stand-in is an unpruned entropy tree with at least 2 rows per leaf. IBk (KNN) defaults are k = 1 with a Euclidean distance that normalises each attribute to its range; our stand-in is min-max scaling followed by 1-NN. The defaults were checked in the Weka documentation [16] (also cited in `notebooks/02_baselines.ipynb`). Each algorithm ran on two row sources per file (64 runs): every row as published (duplicates kept, values untouched), which is closest to the paper's input, and data v1 (deduplicated). **Why.** It tests whether we reproduce the paper's results before we try to improve on them. We compare precision with precision and recall with recall, never the paper's precision with our F1. These scores are never used to choose a model. **Hypotheses and thresholds.** On the published rows, at least 12 of the 16 A1 values of the paper's Figure 2 are matched within +-0.05, for each algorithm. Because duplicates can put a copy of a flow in the training folds, removing them lowers 10-fold macro F1 in at least 12 of 16 files. **Result.** Reproduction supported: our C4.5 approximation matches all 16 A1 values (largest difference 0.021, close to the +-0.01 reading error of the chart), and 1-NN matches 15 of 16 (the miss is VPN recall at 120 s: 0.803 vs 0.860). The duplicate hypothesis is not supported (6 of 16 files lower for the tree, 2 of 16 for 1-NN).

A1, paper (Figure 2, read to +-0.01) vs ours (`diff` = ours - paper):

algorithm	class	timeout	metric	paper	paper_source	ours_published	ours_v1	diff_published	diff_v1
C4.5	Non-VPN	15	precision	0.906	chart +-0.01 and text	0.886	0.882	-0.02	-0.024
C4.5	Non-VPN	30	precision	0.885	chart +-0.01	0.866	0.866	-0.019	-0.019
C4.5	Non-VPN	60	precision	0.864	chart +-0.01	0.86	0.876	-0.004	0.012
C4.5	Non-VPN	120	precision	0.887	chart +-0.01 and text	0.87	0.863	-0.017	-0.024
C4.5	Non-VPN	15	recall	0.877	chart +-0.01	0.882	0.903	0.005	0.026
C4.5	Non-VPN	30	recall	0.846	chart +-0.01	0.857	0.886	0.011	0.04
C4.5	Non-VPN	60	recall	0.888	chart +-0.01	0.889	0.898	0.001	0.01
C4.5	Non-VPN	120	recall	0.843	chart +-0.01	0.853	0.89	0.01	0.047
C4.5	VPN	15	precision	0.89	chart +-0.01 and text	0.892	0.908	0.002	0.018
C4.5	VPN	30	precision	0.868	chart +-0.01	0.874	0.888	0.006	0.02
C4.5	VPN	60	precision	0.856	chart +-0.01	0.857	0.867	0.001	0.011
C4.5	VPN	120	precision	0.863	chart +-0.01 and text	0.868	0.877	0.005	0.014
C4.5	VPN	15	recall	0.917	chart +-0.01	0.896	0.888	-0.021	-0.029
C4.5	VPN	30	recall	0.901	chart +-0.01	0.881	0.867	-0.02	-0.034
C4.5	VPN	60	recall	0.827	chart +-0.01	0.821	0.84	-0.006	0.013
C4.5	VPN	120	recall	0.902	chart +-0.01	0.884	0.849	-0.018	-0.053
KNN	Non-VPN	15	precision	0.846	chart +-0.01 and text	0.821	0.846	-0.025	0.0
KNN	Non-VPN	30	precision	0.84	chart +-0.01	0.805	0.83	-0.035	-0.01
KNN	Non-VPN	60	precision	0.838	chart +-0.01	0.831	0.854	-0.007	0.016

algorithm	class	timeout	metric	paper	paper_source	ours_published	ours_v1	diff_published	diff_v1
KNN	Non-VPN	120	precision	0.837	chart +-0.01 and text	0.794	0.83	-0.043	-0.007
KNN	Non-VPN	15	recall	0.832	chart +-0.01	0.857	0.848	0.025	0.016
KNN	Non-VPN	30	recall	0.808	chart +-0.01	0.839	0.831	0.031	0.023
KNN	Non-VPN	60	recall	0.851	chart +-0.01	0.857	0.849	0.006	-0.002
KNN	Non-VPN	120	recall	0.787	chart +-0.01	0.827	0.825	0.04	0.038
KNN	VPN	15	precision	0.848	chart +-0.01 and text	0.863	0.859	0.015	0.011
KNN	VPN	30	precision	0.835	chart +-0.01	0.85	0.836	0.015	0.001
KNN	VPN	60	precision	0.812	chart +-0.01	0.816	0.811	0.004	-0.001
KNN	VPN	120	precision	0.815	chart +-0.01 and text	0.836	0.813	0.021	-0.002
KNN	VPN	15	recall	0.861	chart +-0.01	0.83	0.857	-0.031	-0.004
KNN	VPN	30	recall	0.862	chart +-0.01	0.818	0.835	-0.044	-0.027
KNN	VPN	60	recall	0.795	chart +-0.01	0.784	0.817	-0.011	0.022
KNN	VPN	120	recall	0.86	chart +-0.01	0.803	0.818	-0.057	-0.042

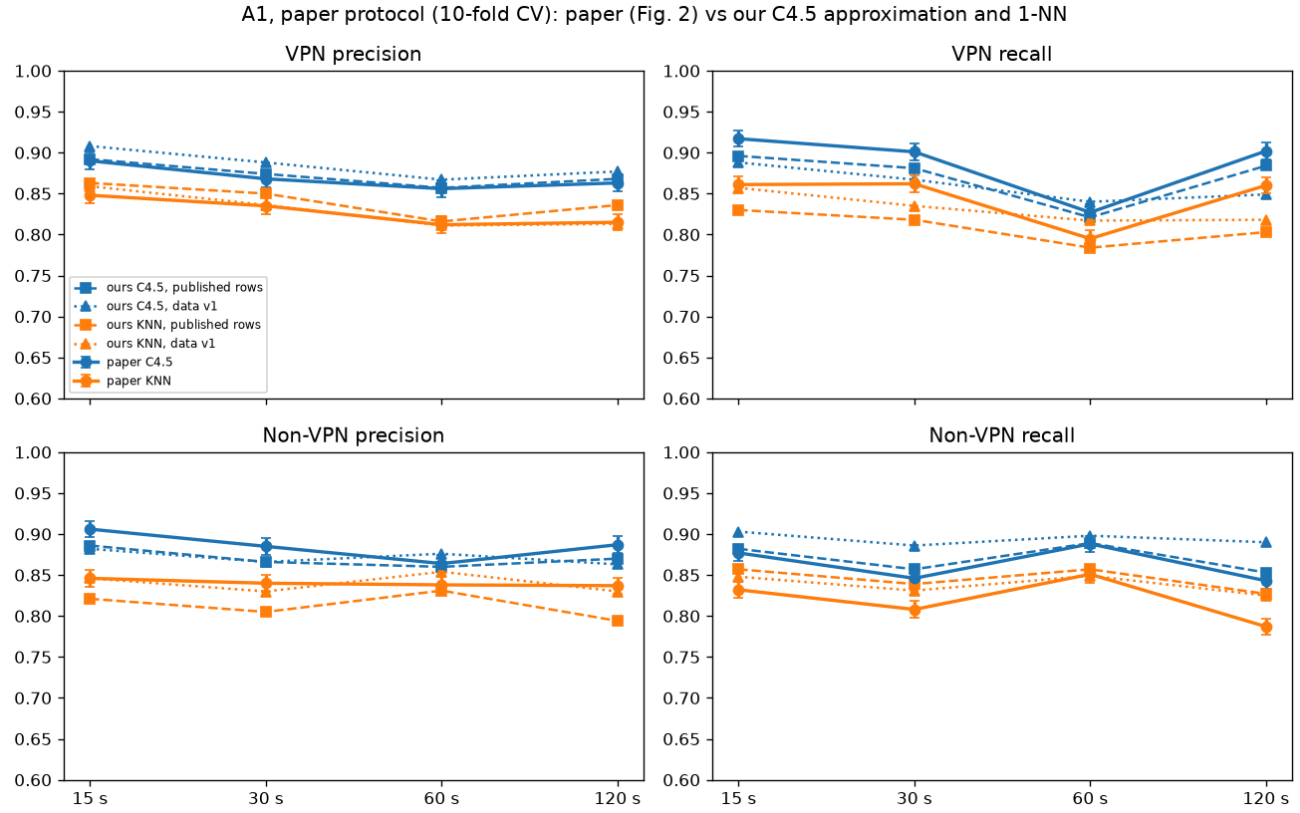


Figure 5. Paper vs ours, A1

The plan asks for the comparison as one table per task. The tables below give our 10-fold precision and recall for every class, both algorithms and all timeouts, on the published rows (closest to the paper's input), with the paper's value in brackets wherever the paper gives one (C = read from its chart, +-0.01; T = stated in its text). The same tables on data v1 are in `reports/baseline_tables.md`.

A2-VPN:

class	metric	C4.5 15 s	C4.5 30 s	C4.5 60 s	C4.5 120 s	KNN 15 s	KNN 30 s	KNN 60 s	KNN 120 s
VPN-BROWSING	precision	0.872	0.878	0.873	0.878	0.830	0.874	0.881	0.795
VPN-BROWSING	recall	0.884	0.916	0.918	0.906	0.846	0.893	0.902	0.888
VPN-CHAT	precision	0.661	0.647	0.568	0.532	0.511	0.490	0.427	0.576
VPN-CHAT	recall	0.687	0.628	0.562	0.473	0.651	0.667	0.658	0.562
VPN-FT	precision	0.792	0.744	0.676	0.649	0.773	0.764	0.714	0.674
VPN-FT	recall	0.770	0.690	0.610	0.605	0.735	0.689	0.648	0.595
VPN-MAIL	precision	0.808	0.891	0.778	0.864	0.766	0.912	0.753	0.859
VPN-MAIL	recall	0.762	0.887	0.715	0.852	0.766	0.947	0.734	0.904
VPN-P2P	precision	0.827	0.831	0.793	0.810	0.736	0.806	0.727	0.771
VPN-P2P	recall	0.863	0.870	0.848	0.862	0.517	0.585	0.527	0.567
VPN-STREAMING	precision	0.922	0.926	0.875	0.838	0.906	0.876	0.838	0.815
VPN-STREAMING	recall	0.891	0.876	0.782	0.752	0.914	0.856	0.787	0.759
VPN-VOIP	precision	0.990	0.992	0.939	0.973	0.990	0.993	0.911	0.984
VPN-VOIP	recall	0.979	0.975	0.914	0.925	0.987	0.981	0.907	0.951
AVERAGE (macro)	precision	0.839 (paper 0.840 T)	0.844	0.786	0.792	0.788	0.816	0.750	0.782
AVERAGE (macro)	recall	0.834	0.835	0.764	0.768	0.774	0.803	0.738	0.747

A2-NonVPN:

class	metric	C4.5 15 s	C4.5 30 s	C4.5 60 s	C4.5 120 s	KNN 15 s	KNN 30 s	KNN 60 s	KNN 120 s
BROWSING	precision	0.904	0.915	0.897	0.928	0.866	0.883	0.865	0.922
BROWSING	recall	0.930	0.934	0.922	0.948	0.880	0.912	0.872	0.932
CHAT	precision	0.813	0.769	0.738	0.656	0.558	0.508	0.483	0.331
CHAT	recall	0.798	0.793	0.753	0.694	0.692	0.662	0.661	0.599
FT	precision	0.810	0.794	0.793	0.796	0.812	0.769	0.847	0.791
FT	recall	0.853	0.824	0.873	0.851	0.670	0.629	0.734	0.571
MAIL	precision	0.849	0.754	0.866	0.859	0.709	0.734	0.762	0.709
MAIL	recall	0.835	0.605	0.786	0.789	0.735	0.716	0.723	0.697
P2P	precision	0.971	0.943	0.948	0.946	0.907	0.897	0.893	0.914
P2P	recall	0.955	0.916	0.919	0.929	0.890	0.883	0.897	0.911
STREAMING	precision	0.837	0.769	0.757	0.768	0.782	0.759	0.655	0.657
STREAMING	recall	0.701	0.664	0.567	0.558	0.685	0.617	0.504	0.524
VOIP	precision	0.992	0.990	0.973	0.989	0.990	0.980	0.968	0.931
VOIP	recall	0.988	0.979	0.946	0.941	0.987	0.977	0.954	0.929
AVERAGE (macro)	precision	0.882 (paper 0.890 T)	0.848	0.853	0.849	0.803	0.790	0.782	0.751

class	metric	C4.5 15 s	C4.5 30 s	C4.5 60 s	C4.5 120 s	KNN 15 s	KNN 30 s	KNN 60 s	KNN 120 s
AVERAGE (macro)	recall	0.866	0.817	0.824	0.816	0.791	0.771	0.764	0.738

B:

class	metric	C4.5 15 s	C4.5 30 s	C4.5 60 s	C4.5 120 s	KNN 15 s	KNN 30 s	KNN 60 s	KNN 120 s
BROWSING	precision	0.771	0.790	0.779	0.800	0.690	0.748	0.746	0.807
BROWSING	recall	0.820	0.831	0.827	0.847	0.700	0.758	0.741	0.794
CHAT	precision	0.677	0.620	0.631	0.510	0.349	0.292	0.336	0.164
CHAT	recall	0.727	0.667	0.667	0.554	0.567	0.536	0.586	0.488
FT	precision	0.717	0.666	0.807	0.687	0.744	0.685	0.809	0.683
FT	recall	0.653	0.614	0.743	0.535	0.622	0.552	0.717	0.487
MAIL	precision	0.633 (paper 0.671 T)	0.617	0.741	0.680 (paper 0.790 T)	0.618	0.712	0.674	0.548
MAIL	recall	0.651	0.617	0.727	0.665	0.643	0.642	0.645	0.616
P2P	precision	0.880	0.851	0.842	0.874	0.749	0.743	0.760	0.818
P2P	recall	0.893	0.845	0.863	0.891	0.732	0.724	0.752	0.804
STREAMING	precision	0.713	0.657	0.496	0.510	0.679	0.630	0.536	0.477
STREAMING	recall	0.670	0.623	0.472	0.495	0.629	0.532	0.440	0.394
VOIP	precision	0.983	0.976	0.912	0.934	0.980	0.974	0.923	0.895
VOIP	recall	0.981	0.971	0.905	0.903	0.983	0.972	0.903	0.895
VPN-BROWSING	precision	0.773 (paper 0.771 T)	0.800	0.797	0.807 (paper 0.809 T)	0.689 (paper 0.691 T)	0.738	0.760	0.742 (paper 0.743 T)
VPN-BROWSING	recall	0.768	0.778	0.802	0.799	0.711	0.759	0.780	0.703
VPN-CHAT	precision	0.616	0.607	0.557	0.534	0.510 (paper 0.501 T)	0.503	0.460	0.466 (paper 0.501 T)
VPN-CHAT	recall	0.571	0.560	0.477	0.444	0.507	0.471	0.442	0.476
VPN-FT	precision	0.759	0.717	0.639	0.661	0.693	0.664	0.584	0.549
VPN-FT	recall	0.712	0.661	0.532	0.539	0.637	0.593	0.528	0.580
VPN-MAIL	precision	0.770 (paper 0.739 T)	0.828	0.702	0.804 (paper 0.786 T)	0.687 (paper 0.688 T)	0.843	0.599	0.820 (paper 0.688 T)
VPN-MAIL	recall	0.670	0.808	0.608	0.802	0.644	0.902	0.554	0.878
VPN-P2P	precision	0.615	0.620	0.547	0.629	0.591	0.666	0.593	0.669
VPN-P2P	recall	0.767	0.771	0.745	0.806	0.428	0.498	0.441	0.514
VPN-STREAMING	precision	0.885	0.871	0.863	0.875	0.836	0.774	0.774	0.713
VPN-STREAMING	recall	0.823	0.812	0.766	0.772	0.848	0.772	0.782	0.703
VPN-VOIP	precision	0.986	0.980	0.893	0.942	0.978	0.963	0.807	0.921
VPN-VOIP	recall	0.973	0.968	0.845	0.889	0.972	0.965	0.809	0.912
AVERAGE (macro)	precision	0.770	0.757	0.729	0.732	0.700	0.710	0.669	0.662
AVERAGE (macro)	recall	0.763	0.752	0.713	0.710	0.687	0.691	0.651	0.660

For A2 and B the paper's Figure 3 draws two unlabelled lines per timeout, which can't be assigned to C4.5 or KNN. So only the values the paper states in its text are compared. The paper doesn't say whether its

"average" is a plain or a class-weighted mean, so both are shown. On the published rows the plain (macro) mean matches it within 0.013 in every case, which suggests the paper used the plain mean.

For averages the paper does not say whether it is a plain or a class-weighted mean (Weka prints the weighted one), so both are shown.

task	algorithm	class	timeout	metric	paper	rows	ours	ours_kind	diff
A2-VPN	C4.5	AVERAGE	15	precision	0.84	published	0.839	macro	-0.001
A2-VPN	C4.5	AVERAGE	15	precision	0.84	published	0.853	weighted	0.013
A2-VPN	C4.5	AVERAGE	15	precision	0.84	v1	0.827	macro	-0.013
A2-VPN	C4.5	AVERAGE	15	precision	0.84	v1	0.846	weighted	0.006
A2-NonVPN	C4.5	AVERAGE	15	precision	0.89	published	0.882	macro	-0.008
A2-NonVPN	C4.5	AVERAGE	15	precision	0.89	published	0.914	weighted	0.024
A2-NonVPN	C4.5	AVERAGE	15	precision	0.89	v1	0.886	macro	-0.004
A2-NonVPN	C4.5	AVERAGE	15	precision	0.89	v1	0.92	weighted	0.03
B	C4.5	VPN-BROWSING	15	precision	0.771	published	0.773	class	0.002
B	C4.5	VPN-BROWSING	15	precision	0.771	v1	0.76	class	-0.011
B	C4.5	VPN-BROWSING	120	precision	0.809	published	0.807	class	-0.002
B	C4.5	VPN-BROWSING	120	precision	0.809	v1	0.76	class	-0.049
B	C4.5	VPN-MAIL	15	precision	0.739	published	0.77	class	0.031
B	C4.5	VPN-MAIL	15	precision	0.739	v1	0.718	class	-0.021
B	C4.5	VPN-MAIL	120	precision	0.786	published	0.804	class	0.018
B	C4.5	VPN-MAIL	120	precision	0.786	v1	0.682	class	-0.104
B	C4.5	MAIL	15	precision	0.671	published	0.633	class	-0.038
B	C4.5	MAIL	15	precision	0.671	v1	0.625	class	-0.046
B	C4.5	MAIL	120	precision	0.79	published	0.68	class	-0.11
B	C4.5	MAIL	120	precision	0.79	v1	0.613	class	-0.177
B	KNN	VPN-BROWSING	15	precision	0.691	published	0.689	class	-0.002
B	KNN	VPN-BROWSING	15	precision	0.691	v1	0.695	class	0.004
B	KNN	VPN-BROWSING	120	precision	0.743	published	0.742	class	-0.001
B	KNN	VPN-BROWSING	120	precision	0.743	v1	0.7	class	-0.043
B	KNN	VPN-CHAT	15	precision	0.501	published	0.51	class	0.009
B	KNN	VPN-CHAT	15	precision	0.501	v1	0.509	class	0.008
B	KNN	VPN-CHAT	120	precision	0.501	published	0.466	class	-0.035
B	KNN	VPN-CHAT	120	precision	0.501	v1	0.494	class	-0.007
B	KNN	VPN-MAIL	15	precision	0.688	published	0.687	class	-0.001
B	KNN	VPN-MAIL	15	precision	0.688	v1	0.633	class	-0.055
B	KNN	VPN-MAIL	120	precision	0.688	published	0.82	class	0.132
B	KNN	VPN-MAIL	120	precision	0.688	v1	0.521	class	-0.167
B	C4.5	AVERAGE_BEST_TIMEOUT	best	precision	0.783	published	0.77	macro	-0.013
B	C4.5	AVERAGE_BEST_TIMEOUT	best	precision	0.783	published	0.808	weighted	0.025

task	algorithm	class	timeout	metric	paper	rows	ours	ours_kind	diff
B	C4.5	AVERAGE_BEST_TIMEOUT	best	precision	0.783	v1	0.774	macro	-0.009
B	C4.5	AVERAGE_BEST_TIMEOUT	best	precision	0.783	v1	0.815	weighted	0.032
B	KNN	AVERAGE_BEST_TIMEOUT	best	precision	0.711	published	0.71	macro	-0.001
B	KNN	AVERAGE_BEST_TIMEOUT	best	precision	0.711	published	0.745	weighted	0.034
B	KNN	AVERAGE_BEST_TIMEOUT	best	precision	0.711	v1	0.707	macro	-0.004
B	KNN	AVERAGE_BEST_TIMEOUT	best	precision	0.711	v1	0.755	weighted	0.044

Duplicates. Removing the exact duplicates changed the 10-fold macro F1 by -0.018 to +0.014 for the tree (mean +0.001) and by -0.017 to +0.040 for 1-NN (mean +0.012):

algo	task	timeout	rows_published	rows_v1	f1_published	f1_v1	v1_minus_published
j48	A1	15	18,758	18,038	0.8888	0.8954	0.0067
j48	A1	30	14,651	13,610	0.8696	0.8766	0.0069
j48	A1	60	15,515	14,677	0.8562	0.8702	0.014
j48	A1	120	10,782	9,524	0.8688	0.8694	0.0007
j48	A2-VPN	15	9,793	9,380	0.836	0.8197	-0.0162
j48	A2-VPN	30	7,734	6,944	0.8389	0.821	-0.0179
j48	A2-VPN	60	6,935	6,523	0.7741	0.7702	-0.0039
j48	A2-VPN	120	5,631	4,648	0.779	0.7719	-0.0071
j48	A2-NonVPN	15	8,965	8,694	0.8731	0.8768	0.0037
j48	A2-NonVPN	30	6,917	6,694	0.8303	0.8359	0.0055
j48	A2-NonVPN	60	8,580	8,206	0.8359	0.8483	0.0123
j48	A2-NonVPN	120	5,151	4,941	0.8294	0.8346	0.0052
j48	B	15	18,758	18,074	0.765	0.7707	0.0057
j48	B	30	14,651	13,638	0.7535	0.7488	-0.0047
j48	B	60	15,515	14,729	0.7185	0.7293	0.0107
j48	B	120	10,782	9,589	0.7183	0.718	-0.0003
ibk	A1	15	18,758	18,038	0.8424	0.8524	0.01
ibk	A1	30	14,651	13,610	0.8276	0.8331	0.0055
ibk	A1	60	15,515	14,677	0.8217	0.8326	0.0109
ibk	A1	120	10,782	9,524	0.8146	0.8218	0.0072
ibk	A2-VPN	15	9,793	9,380	0.7766	0.793	0.0164
ibk	A2-VPN	30	7,734	6,944	0.8047	0.81	0.0054
ibk	A2-VPN	60	6,935	6,523	0.7377	0.7552	0.0175
ibk	A2-VPN	120	5,631	4,648	0.7611	0.7441	-0.017
ibk	A2-NonVPN	15	8,965	8,694	0.7949	0.8118	0.0169
ibk	A2-NonVPN	30	6,917	6,694	0.7769	0.7989	0.022
ibk	A2-NonVPN	60	8,580	8,206	0.7687	0.7899	0.0213
ibk	A2-NonVPN	120	5,151	4,941	0.7349	0.7751	0.0402
ibk	B	15	18,758	18,074	0.69	0.7071	0.0171
ibk	B	30	14,651	13,638	0.6958	0.6931	-0.0027
ibk	B	60	15,515	14,729	0.6561	0.6697	0.0137
ibk	B	120	10,782	9,589	0.6531	0.661	0.0079

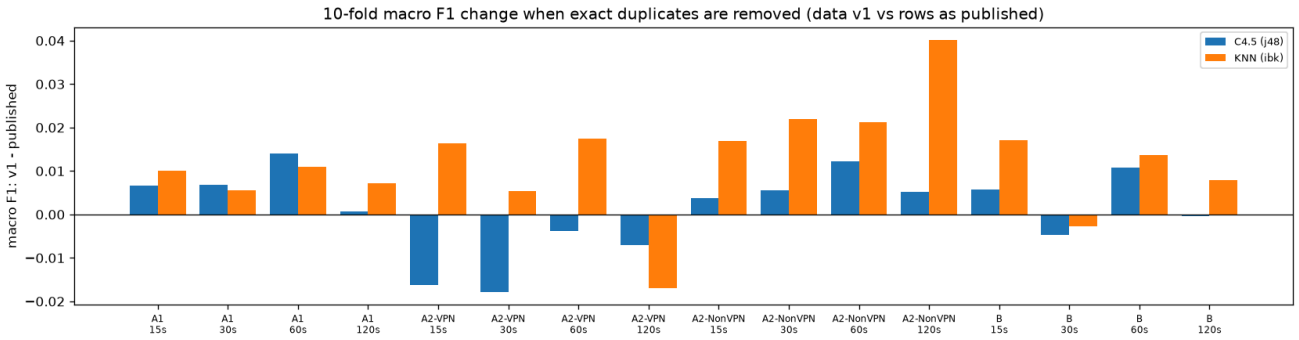


Figure 6. Duplicate effect

The reason the hypothesis failed is in the rows themselves. The removed duplicates are not copies of ordinary flows. Between 39% and 92% of them have an identical twin with a different label, against 2-10% of all rows, and their median flow has 0-2 packets (all flows: 8-43). They are near-empty flows that timing features can't tell apart. Keeping every copy multiplies errors that no model can avoid, which outweighs the advantage of having a test flow's copy in the training data:

Share of removed rows whose features also occur with another label, and packets per flow. A1 120 s packet counts use its published (corrupt) flowPktsPerSecond.

task	timeout	duplicates removed	removed with other-label twin	all rows with other-label twin	median packets, removed	median packets, all
A1	15	720	0.761	0.032	0.0	32.0
A1	30	1,041	0.51	0.039	2.0	22.0
A1	60	838	0.766	0.047	0.0	16.0
A1	120	1,257	0.434	0.056	0.0	0.0
A2-VPN	15	413	0.823	0.04	0.0	27.0
A2-VPN	30	790	0.395	0.045	2.0	17.0
A2-VPN	60	412	0.748	0.053	0.0	8.0
A2-VPN	120	983	0.45	0.098	2.0	8.0
A2-NonVPN	15	271	0.742	0.024	0.0	43.0
A2-NonVPN	30	223	0.897	0.03	0.0	26.0
A2-NonVPN	60	374	0.805	0.039	0.0	23.0
A2-NonVPN	120	210	0.919	0.039	0.0	17.0
B	15	684	0.804	0.035	0.0	32.0
B	30	1,013	0.529	0.042	2.0	22.0
B	60	786	0.818	0.053	0.0	16.0
B	120	1,193	0.562	0.078	2.0	12.0

The duplicate rule of data v1 stays: in our own split it guarantees that no flow is scored after being trained on, and this benchmark shows it changes 10-fold scores by only about +-0.02.

4.4 Storage after the baselines

Every official run saves its trained model (the plan's rule). After the first 147 runs (80 baselines, 64 benchmark, 2 pilot, 1 setup check), the MLflow store held 137.8 MB of artifacts and a 5.8 MB database (275.2 MB and 10.4 MB after the timing rerun of all 144 configs); the largest single run is 4.3 MB (SVM). Per model type:

kind	runs	total_MB	per_run_max_MB	model_file_max_MB
knn (holdout)	16	23.075	2.475	2.319
knn_minmax (paper_10fold)	32	66.742	3.596	3.438
logreg (holdout)	16	1.64	0.167	0.005
majority (holdout)	16	1.395	0.118	0.001
svm_rbf (holdout)	16	27.418	4.345	4.188
tree (holdout)	16	5.457	0.764	0.579
tree (paper_10fold)	32	11.674	0.817	0.629

Whole store now (after the rerun): artifacts 275.2 MB, mlflow.db 10.4 MB.

Tree ensembles in Phase 3 will be much larger: a 300-tree Random Forest, measured on the largest training set (B, 15 s), is a 171.9 MB file (22.4 MB with joblib compression level 3) [E03].

4.5 Phase 3: how the improvement experiments were run

Phase 3 (E04-E17) starts from the Phase 2 baselines and changes **one thing at a time** against the **current best** model of each task: the highest 15 s validation macro F1 so far, whatever the margin. Every experiment runs first at 15 s for all four tasks. A model is a **winner**, and is rerun at 30, 60 and 120 s with its 15 s settings, only if it beats the task's current best by at least 0.03; 0.03 is above the largest Phase 2 cross-validation standard deviation (0.025), so a smaller gain could be noise from the split. The rule was fixed on 28 Oct, before any Phase 3 run, and written into every experiment entry before it ran. All Phase 3 runs use data v1, the same dependency lock file and one thread per model; every model is saved with joblib compression level 3. Every table below is copied unchanged from `reports/phase3_tables.md`, generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`; the figures come from notebooks 03 and 04 (`scripts/make_notebooks.py`). Scores are validation scores of the fixed 70/15/15 split (seed 42); the test set is not used in Phase 3.

4.6 E04: KNN with a signed log transform

What. KNN (k = 5, as in E03) with a signed log, sign(x)·log1p(|x|), applied inside the pipeline before scaling, once with StandardScaler and once with RobustScaler (Section 3.7). **Why.** KNN compares flows by distance, and a few very large timing values (up to about 1.2e8 microseconds) dominate every distance on the raw features. **Threshold (pre-stated).** (a) the better variant beats the E03 KNN by at least 0.02 in at least 3 of 4 tasks; (b) the two scalers are within 0.02 of each other in at least 3 of 4 tasks.

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E04-knn_log_robust	0.8929	0.8874 +- 0.0042	E02-tree	0.8901	0.0028	no
A1	E04-knn_log_standard	0.9048	0.9033 +- 0.0046	E02-tree	0.8901	0.0146	no
A2-VPN	E04-knn_log_robust	0.8372	0.8185 +- 0.0080	E02-tree	0.8247	0.0125	no
A2-VPN	E04-knn_log_standard	0.8544	0.8336 +- 0.0103	E02-tree	0.8247	0.0296	no
A2-NonVPN	E04-knn_log_robust	0.854	0.8688 +- 0.0050	E02-tree	0.8294	0.0247	no
A2-NonVPN	E04-knn_log_standard	0.867	0.8861 +- 0.0068	E02-tree	0.8294	0.0376	yes
B	E04-knn_log_robust	0.742	0.7555 +- 0.0094	E02-tree	0.7585	-0.0166	no

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
B	E04-knn_log_standard	0.7644	0.7798 +- 0.0078	E02-tree	0.7585	0.0059	no

Result. The signed log lifts KNN by 0.080-0.104 in every task, and StandardScaler beats RobustScaler in every task (by 0.0118-0.0224). The signed-log KNN also passes the E02 tree in all four tasks and becomes the current best; only A2-NonVPN (+0.0376) is a winner (A2-VPN misses the margin by 0.0004). Rerun at the other timeouts, it stays the best A2-NonVPN run at each:

task	timeout	model	val macro F1	CV macro F1	best other run	its F1	difference
A2-NonVPN	15 s	E04-knn_log_standard	0.867	0.8861 +- 0.0068	E04-knn_log_robust	0.854	0.0129
A2-NonVPN	30 s	E04-knn_log_standard	0.8523	0.8487 +- 0.0067	E02-tree	0.8283	0.024
A2-NonVPN	60 s	E04-knn_log_standard	0.859	0.8638 +- 0.0108	E02-tree	0.8246	0.0344
A2-NonVPN	120 s	E04-knn_log_standard	0.8491	0.8243 +- 0.0252	E02-tree	0.8393	0.0098

Verdict. (a) supported, (b) supported. [E04]

4.7 E05: Random Forest

What. A 300-tree Random Forest with default settings. **Why.** The E02 tree is unpruned (depth 16-35) and fits noise; averaging 300 trees grown on bootstrap samples lowers that variance. **Threshold.** (a) the forest beats the E02 tree by at least 0.03 in at least 3 of 4 tasks; (b) it beats the current best (the E04 KNN) in at least 2 of 4 tasks.

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E05-rf	0.9203	0.9172 +- 0.0050	E04-knn_log_standard	0.9048	0.0156	no
A2-VPN	E05-rf	0.8679	0.8637 +- 0.0066	E04-knn_log_standard	0.8544	0.0135	no
A2-NonVPN	E05-rf	0.8941	0.9048 +- 0.0049	E04-knn_log_standard	0.867	0.0271	no
B	E05-rf	0.8147	0.8102 +- 0.0090	E04-knn_log_standard	0.7644	0.0503	yes

Result. The forest is the current best in all four tasks (+0.0135 to +0.0503 over the E04 KNN; +0.0302 to +0.0647 over the tree). Only B is a winner; its reruns:

task	timeout	model	val macro F1	CV macro F1	best other run	its F1	difference
B	15 s	E05-rf	0.8147	0.8102 +- 0.0090	E04-knn_log_standard	0.7644	0.0503
B	30 s	E05-rf	0.8114	0.7979 +- 0.0119	E02-tree	0.7476	0.0638
B	60 s	E05-rf	0.7771	0.7656 +- 0.0136	E02-tree	0.7058	0.0713
B	120 s	E05-rf	0.7835	0.7562 +- 0.0083	E02-tree	0.7158	0.0677

Verdict. (a) and (b) supported. The forest is the largest model so far: 171.9 MiB uncompressed for B at 15 s, 22.4 MiB as a compressed file. [E05]

4.8 E06: gradient boosting (XGBoost and LightGBM)

What. XGBoost and LightGBM with default settings. **Why.** Boosted trees correct the errors of earlier trees, which should help the classes the forest still confuses. **Threshold.** (a) the better booster beats the forest in at least 2 of 4 tasks; (b) the two are within 0.02 of each other in at least 3 of 4 tasks.

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E06-lgbm	0.9181	0.9175 +- 0.0042	E05-rf	0.9203	-0.0022	no
A1	E06-xgb	0.9271	0.9251 +- 0.0046	E05-rf	0.9203	0.0067	no
A2-VPN	E06-lgbm	0.8694	0.8827 +- 0.0080	E05-rf	0.8679	0.0015	no
A2-VPN	E06-xgb	0.8758	0.8799 +- 0.0025	E05-rf	0.8679	0.0079	no
A2-NonVPN	E06-lgbm	0.8983	0.9224 +- 0.0097	E05-rf	0.8941	0.0042	no
A2-NonVPN	E06-xgb	0.9015	0.9237 +- 0.0117	E05-rf	0.8941	0.0075	no
B	E06-lgbm	0.8201	0.8305 +- 0.0088	E05-rf	0.8147	0.0054	no
B	E06-xgb	0.8303	0.8334 +- 0.0053	E05-rf	0.8147	0.0156	no

Result. Default XGBoost is the current best in all four tasks (+0.0067 to +0.0156 over the forest); LightGBM is 0.0032-0.0102 below it. No winner. **Verdict.** (a) and (b) supported. Default XGBoost is also one of the cheapest models to run: 1.13-11.76 ms per 1,000 flows, against 21.91-27.28 ms for the forest. The figures show validation macro F1 with cross-validation mean +- std for every E02-E06 model at 15 s, the F1 of every class, and macro F1 against training and prediction time. [E06]

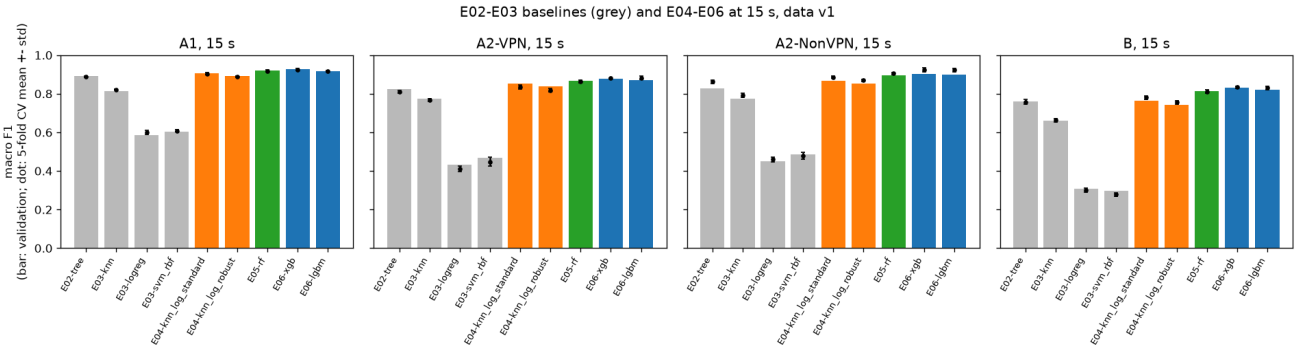


Figure 7. E02-E06 at 15 s

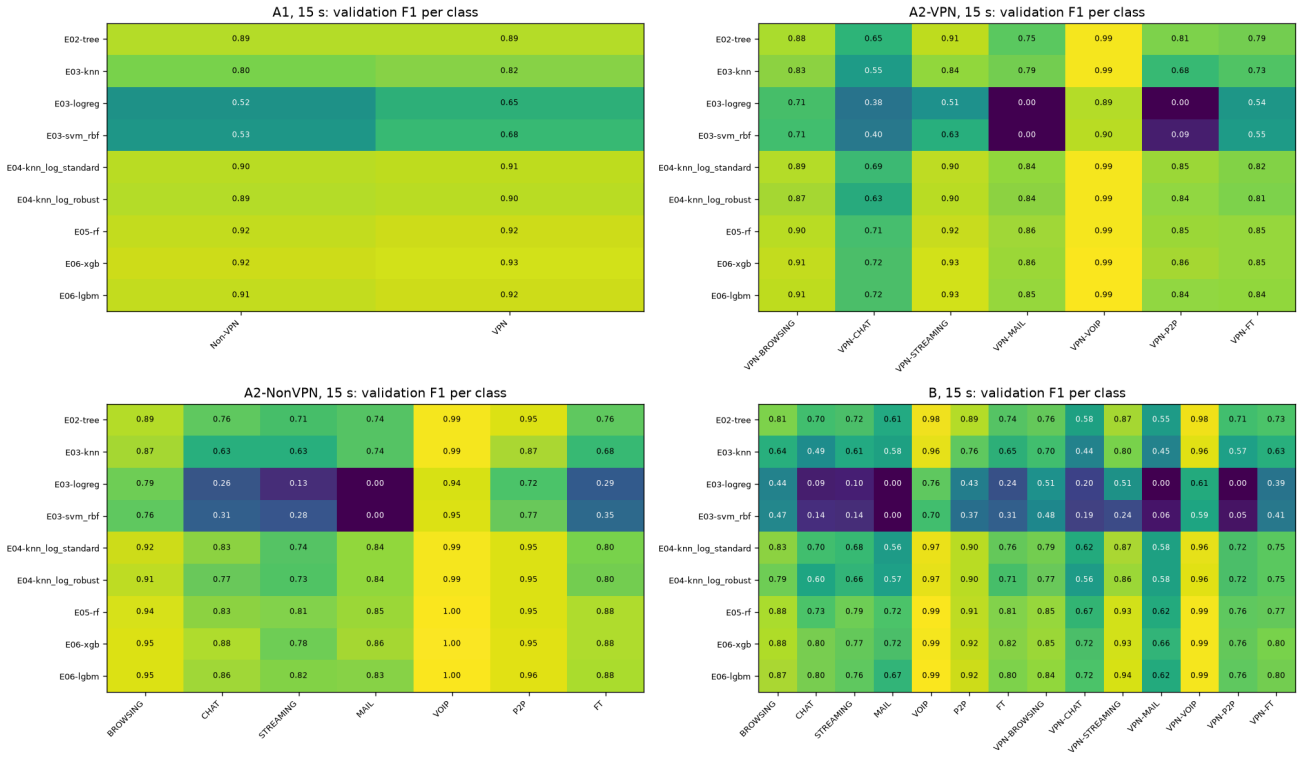


Figure 8. Per-class F1, E02-E06

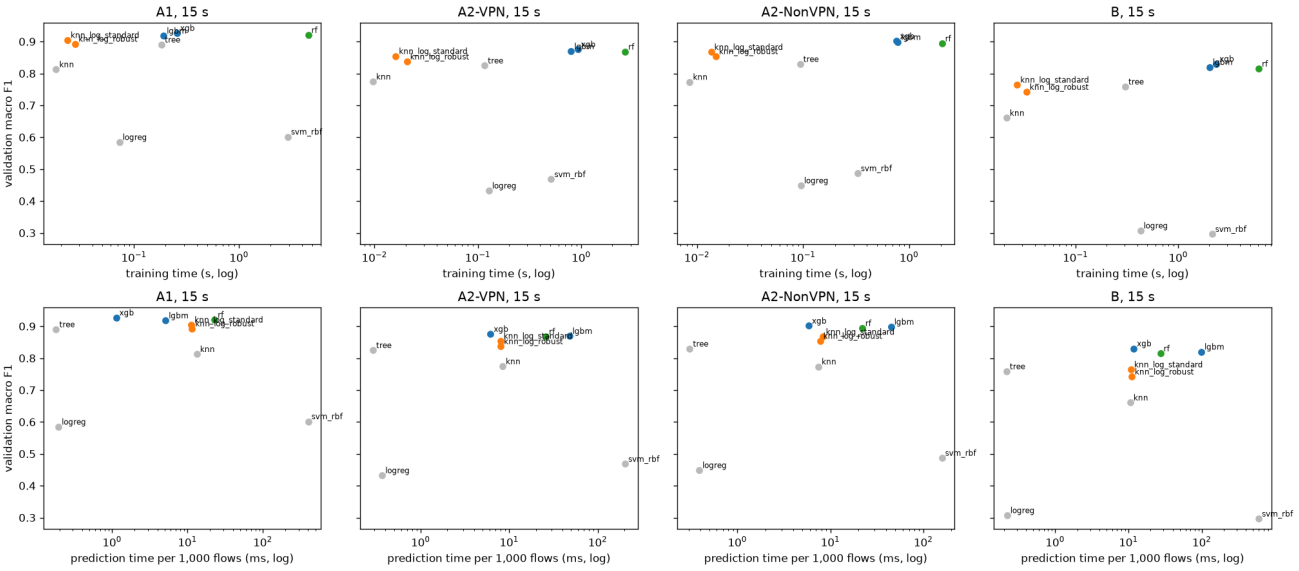


Figure 9. Macro F1 against cost, E02-E06

4.9 E07: hyperparameter tuning with Optuna

What. For each task the best 2 of Random Forest, XGBoost and LightGBM, plus the signed-log KNN, each tuned with the same budget: 50 Optuna trials (TPE sampler, seed 42), each trial scored by 5-fold cross-validation macro F1 on the training rows only. The search ranges were fixed in the plan before running. The best trial is retrained and scored on validation like any other run. **Why.** Defaults are general-purpose; giving every candidate the same budget means no model wins only because it was tuned more. **Threshold.** (a) tuned above untuned in at least 8 of 12 searches; (b) the best E07 model beats the current best (default XGBoost) by at least 0.01 in at least 2 of 4 tasks; (c) the tuned KNN stays below the best tuned tree ensemble in all four tasks.

task	ranking (15 s val macro F1)	tuned in E07	KNN preprocessing for E07
A1	xgb 0.9271, rf 0.9203, lgbm 0.9181	xgb, rf	knn_log_standard 0.9048

task	ranking (15 s val macro F1)	tuned in E07	KNN preprocessing for E07
A2-VPN	xgb 0.8758, lgbm 0.8694, rf 0.8679	xgb, lgbm	knn_log_standard 0.8544
A2-NonVPN	xgb 0.9015, lgbm 0.8983, rf 0.8941	xgb, lgbm	knn_log_standard 0.8670
B	xgb 0.8303, lgbm 0.8201, rf 0.8147	xgb, lgbm	knn_log_standard 0.7644

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E07-knn_log_standard	0.9163	0.9122 +- 0.0047	E06-xgb	0.9271	-0.0108	no
A1	E07-rf	0.9181	0.9209 +- 0.0025	E06-xgb	0.9271	-0.009	no
A1	E07-xgb	0.9259	0.9293 +- 0.0065	E06-xgb	0.9271	-0.0011	no
A2-VPN	E07-knn_log_standard	0.8625	0.8575 +- 0.0048	E06-xgb	0.8758	-0.0133	no
A2-VPN	E07-lgbm	0.8768	0.8902 +- 0.0023	E06-xgb	0.8758	0.001	no
A2-VPN	E07-xgb	0.8661	0.8838 +- 0.0011	E06-xgb	0.8758	-0.0097	no
A2-NonVPN	E07-knn_log_standard	0.8929	0.9060 +- 0.0071	E06-xgb	0.9015	-0.0086	no
A2-NonVPN	E07-lgbm	0.9028	0.9329 +- 0.0072	E06-xgb	0.9015	0.0013	no
A2-NonVPN	E07-xgb	0.9025	0.9294 +- 0.0067	E06-xgb	0.9015	0.001	no
B	E07-knn_log_standard	0.8006	0.8044 +- 0.0111	E06-xgb	0.8303	-0.0297	no
B	E07-lgbm	0.8352	0.8447 +- 0.0058	E06-xgb	0.8303	0.0049	no
B	E07-xgb	0.8428	0.8405 +- 0.0056	E06-xgb	0.8303	0.0125	no

Result. Tuning raised the cross-validation score in all 12 searches but the validation score in only 9: the best of 50 cross-validation scores is optimistic. The largest gains are for KNN (B +0.0361, A2-NonVPN +0.0260); the ensembles gain at most +0.0151. Only B improves by more than 0.01 (tuned XGBoost 0.8428, +0.0125). No winner. In every search the retrained model's cross-validation score equals its best trial's exactly. **Verdict.** (a) supported (9 of 12), (b) not supported (1 of 4 tasks), (c) supported. [E07]

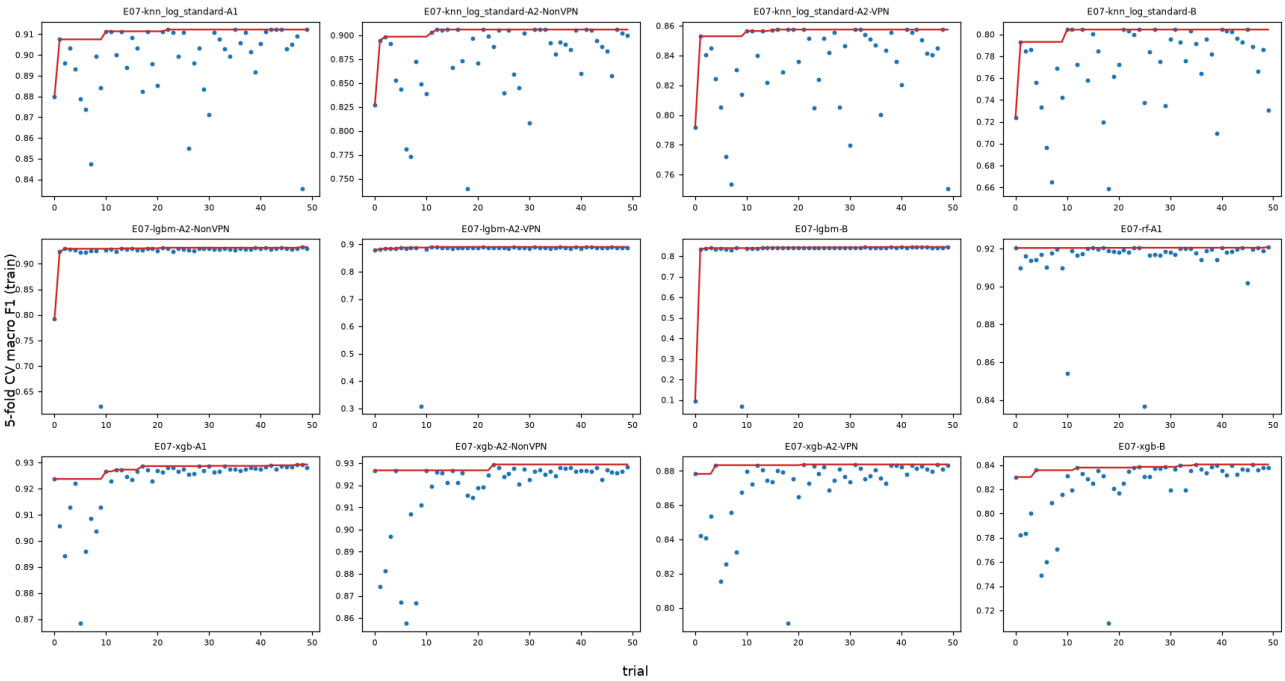


Figure 10. E07 Optuna histories

4.10 E08: class balancing

What. On the current best model per task, a ladder: none, then class weights, then random oversampling, then SMOTE (inside the pipeline, so only training folds are resampled); a step runs only if every earlier step helped. **Why.** Three tasks are imbalanced (largest to smallest class 5.79:1 in A2-VPN, 11.85:1 in A2-NonVPN and B), and macro F1 weights every class equally. **Threshold.** (a) the best step beats "none" by at least 0.01 in at least 2 of the 3 imbalanced tasks; (b) in A1 no step moves macro F1 by more than 0.01; (c) the best step raises the F1 of the weakest class in at least 2 of the 3 imbalanced tasks.

task	step	run	val macro F1	change vs none	F1 of weakest class without balancing	helped	next
A1	none	E06-xgb	0.9271	0.0	Non-VPN 0.9240		
A1	class_weight	E08-xgb_class_weight	0.9237	-0.0034	Non-VPN 0.9202	no	stop: this step did not help
A2-VPN	none	E07-lgbm	0.8768	0.0	VPN-CHAT 0.7343		
A2-VPN	class_weight	E08-lgbm_class_weight	0.883	0.0062	VPN-CHAT 0.7463	yes	
A2-VPN	oversample	E08-lgbm_oversample	0.883	0.0062	VPN-CHAT 0.7477	no	stop: this step did not help
A2-NonVPN	none	E07-lgbm	0.9028	0.0	STREAMING 0.8095		
A2-NonVPN	class_weight	E08-lgbm_class_weight	0.9049	0.0022	STREAMING 0.8189	yes	
A2-NonVPN	oversample	E08-lgbm_oversample	0.9075	0.0047	STREAMING 0.8372	yes	
A2-NonVPN	smote	E08-lgbm_smote	0.9046	0.0019	STREAMING 0.8217	no	stop: this step did not help
B	none	E07-xgb	0.8428	0.0	VPN-MAIL 0.6496		
B	class_weight	E08-xgb_class_weight	0.8364	-0.0064	VPN-MAIL 0.6290	no	stop: this step did not help

Result. Every change is within +-0.0064. Balancing helps a little in A2-VPN (class weights +0.0062) and A2-NonVPN (random oversampling +0.0047) and hurts in A1 and B. **Verdict.** (a) not supported, (b) supported, (c)

supported. The weak classes (VPN-CHAT, STREAMING, VPN-MAIL) seem to fail because their flows look like those of other classes, not because there are too few of them; E08 does not test this directly. [E08]

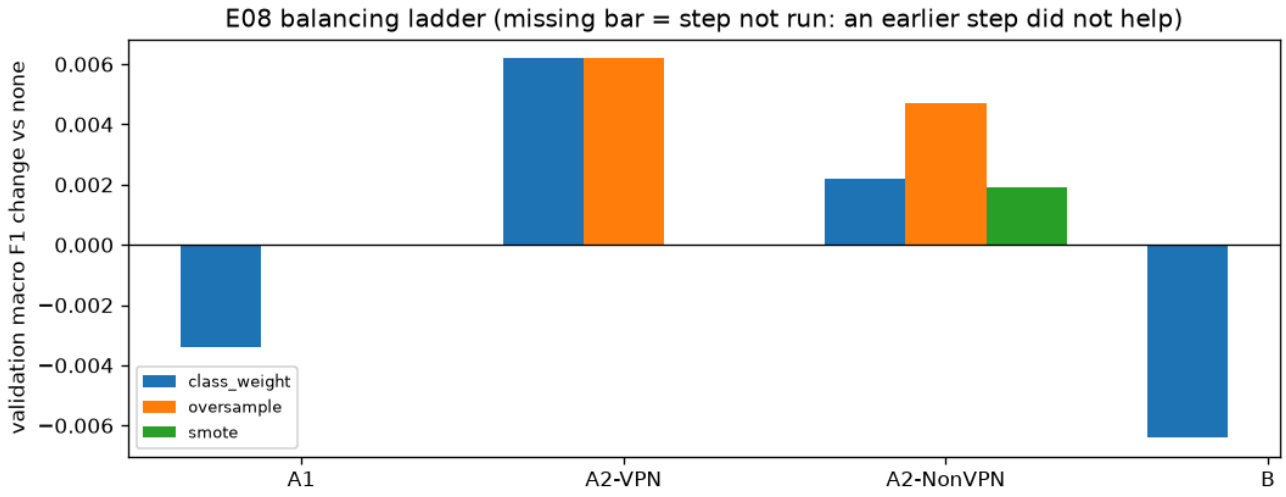


Figure 11. E08 balancing ladder

4.11 E10: task B as one 14-class model or two steps

What. A two-step model for task B: a VPN / non-VPN model routes each flow to one of two 7-class models. All three models use the current best B model type and settings (tuned XGBoost), and all are trained on B's training rows only. They are never trained on the A1 split, because A1 and B hold the same flows split differently. **Why.** Task B mixes two questions; separating them could help. **Threshold.** (a) the two-step score is within 0.01 of the direct model; (b) its first step makes fewer VPN / non-VPN errors than the direct model (180); (c) more than half of its errors come from a wrong first step.

structure	run	val macro F1	CV macro F1	val accuracy	rows	errors	group errors (VPN / non-VPN wrong)	type errors (right group, wrong type)	group accuracy	share of errors from the group
direct 14-class	E07-xgb	0.8428	0.8405 +- 0.0056	0.8794	2,711	327	180	147	0.9336	0.5505
two-step	E10-xgb_two_step	0.8318	0.8274 +- 0.0083	0.8727	2,711	345	191	154	0.9295	0.5536

Result. The two-step model scores 0.0110 lower, makes more group errors (191 vs 180), and 55.36% of its errors come from the first step. The direct model also makes 55.05% of its errors across the VPN / non-VPN line: telling VPN from non-VPN is the hard part either way. **Verdict.** (a) not supported, (b) not supported, (c) supported. Task B stays a direct 14-class model. [E10]

4.12 E17: the same tuning budget for every method type

What. The same 50-trial search as E07 for every method type not yet tuned: decision tree, logistic regression and SVM-RBF in all four tasks, Random Forest in A2-VPN, A2-NonVPN and B, and LightGBM in A1 (16 searches). Logistic regression and SVM may choose the signed log inside the search. **Why.** Without it, a comparison of method types would mix tuned and untuned models, the weakness we point out in earlier studies. Added on 28 Oct after E10, with the ranges agreed before running. **Threshold.** (a) tuned above untuned in at least 12 of 16 searches; (b) the best run per task is still XGBoost or LightGBM in at least 3 of 4 tasks; (c) the signed log is chosen in at least 6 of the 8 logistic-regression and SVM searches.

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E17-lgbm	0.9308	0.9320 +- 0.0028	E06-xgb	0.9271	0.0037	no

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E17-logreg	0.6517	0.6762 +- 0.0051	E06-xgb	0.9271	-0.2754	no
A1	E17-svm_rbf	0.9259	0.9187 +- 0.0044	E06-xgb	0.9271	-0.0011	no
A1	E17-tree	0.8905	0.8915 +- 0.0043	E06-xgb	0.9271	-0.0366	no
A2-VPN	E17-logreg	0.7263	0.7344 +- 0.0074	E08-lgbm_class_weight	0.883	-0.1566	no
A2-VPN	E17-rf	0.8704	0.8707 +- 0.0069	E08-lgbm_class_weight	0.883	-0.0126	no
A2-VPN	E17-svm_rbf	0.8707	0.8668 +- 0.0074	E08-lgbm_class_weight	0.883	-0.0123	no
A2-VPN	E17-tree	0.8292	0.8129 +- 0.0071	E08-lgbm_class_weight	0.883	-0.0538	no
A2-NonVPN	E17-logreg	0.8002	0.8138 +- 0.0120	E08-lgbm_oversample	0.9075	-0.1073	no
A2-NonVPN	E17-rf	0.8829	0.9063 +- 0.0050	E08-lgbm_oversample	0.9075	-0.0246	no
A2-NonVPN	E17-svm_rbf	0.9058	0.9256 +- 0.0042	E08-lgbm_oversample	0.9075	-0.0017	no
A2-NonVPN	E17-tree	0.8294	0.8619 +- 0.0086	E08-lgbm_oversample	0.9075	-0.0781	no
B	E17-logreg	0.5862	0.5909 +- 0.0067	E07-xgb	0.8428	-0.2566	no
B	E17-rf	0.8167	0.8270 +- 0.0092	E07-xgb	0.8428	-0.0261	no
B	E17-svm_rbf	0.8242	0.8228 +- 0.0059	E07-xgb	0.8428	-0.0186	no
B	E17-tree	0.7585	0.7575 +- 0.0132	E07-xgb	0.8428	-0.0843	no

Result. SVM-RBF gains the most (+0.3243 to +0.5277), and every logistic-regression and SVM search chose the signed log. The tuned SVM ends 0.0017-0.0186 below each task's best. Logistic regression gains +0.0670 to +0.3515 but stays at 0.5862-0.8002. The tree (0.0000 to +0.0044) and the forest (-0.0111 to +0.0025) barely change. Tuned LightGBM is the new A1 best (0.9308). No winner. **Verdict.** (a) supported (13 of 16), (b) supported (4 of 4), (c) supported (8 of 8). The A2-NonVPN logistic regression stops at its fixed 1,000 iterations before converging (chosen C = 985); this is recorded as a limitation and was not changed after the run. [E17]

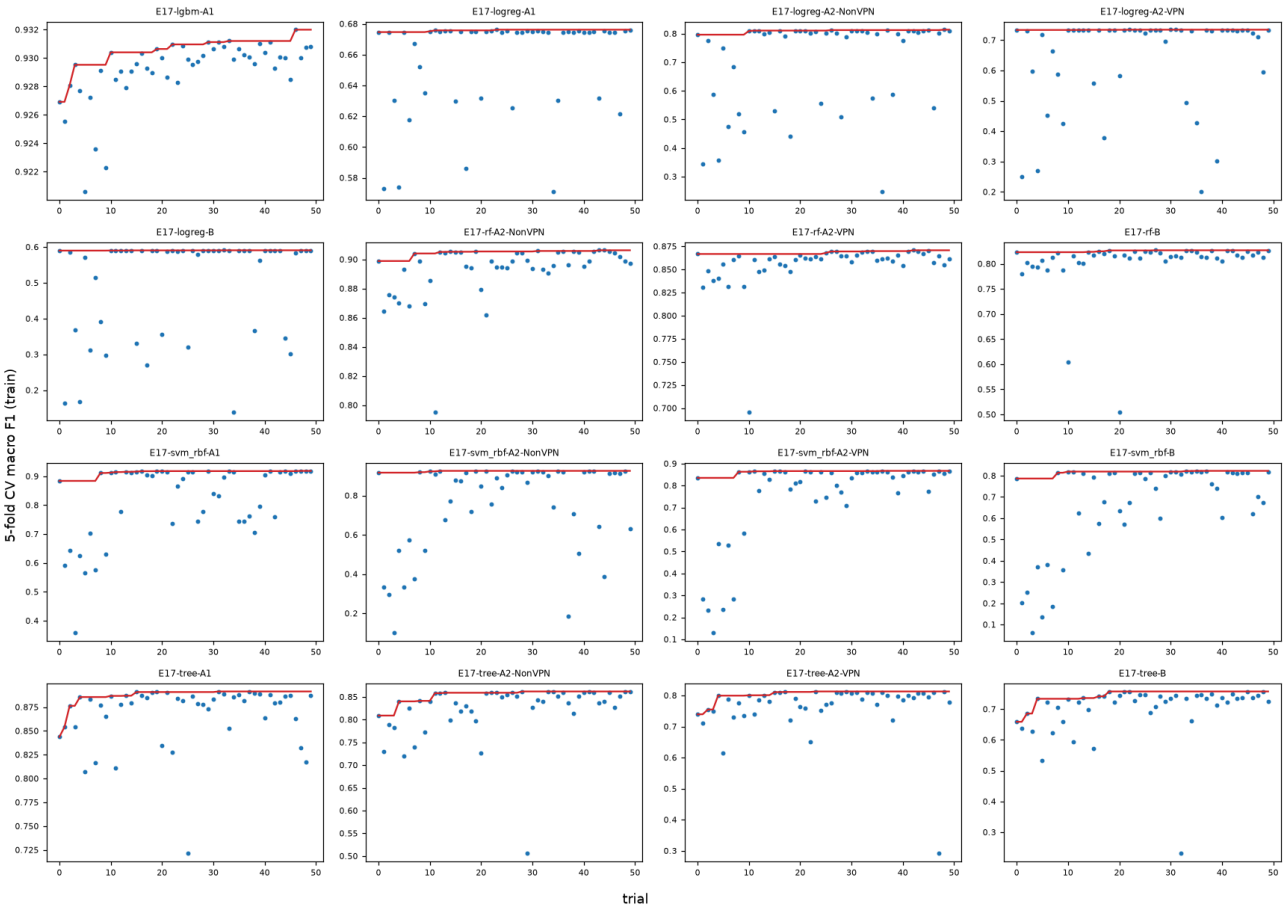


Figure 12. E17 Optuna histories

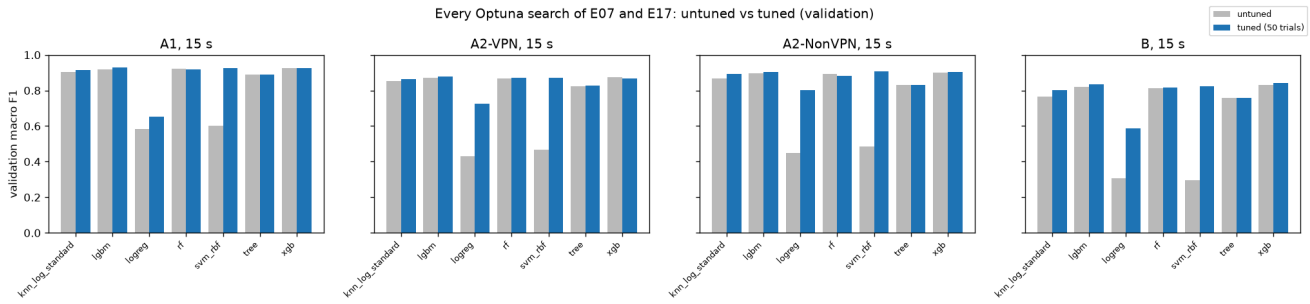


Figure 13. Untuned vs tuned, E07 and E17

4.13 Final choice per task and the comparison set

The final configuration per task is the run with the highest validation macro F1 among all model-selection runs (E02-E17, any timeout), decided on 28 Oct after E10. This replaces the rule set before Phase 3, under which any gap below 0.03 was a tie broken in favour of the simpler model. The cross-validation standard deviations of the headline models (0.0028-0.0058) are far below 0.03, and under the old rule KNN or the SVM would have been chosen at 0.0146-0.0286 below the best. This is a deviation from the pre-set rule and is reported as one (Section 3.8). The models the old rule would pick are also scored on the test set in Phase 4 (E12), so the cost of either rule is visible. The only fallback is a saved model file above 50 MB; none of the four is.

task	run	method type	timeout	val macro F1	next best	next best F1	model file MiB	skipped for file size
A1	E17-lgbm-A1-15s	LightGBM	15	0.9308	E06-xgb-A1-15s	0.9271	7.09	none
A2-VPN	E08-lgbm_class_weight-A2-VPN-15s	LightGBM	15	0.883	E08-lgbm_oversample-A2-VPN-15s	0.883	25.12	none

task	run	method type	timeout	val macro F1	next best	next best F1	model file MiB	skipped for file size
A2-NonVPN	E08-lgbm_oversample-A2-NonVPN-15s	LightGBM	15	0.9075	E17-svm_rbf-A2-NonVPN-15s	0.9058	4.32	none
B	E07-xgb-B-15s	XGBoost	15	0.8428	E08-xgb_class_weight-B-15s	0.8364	4.31	none

task	run	method type	val macro F1	below the best by	runs within 0.03	in comparison set
A1	E04-knn_log_standard-A1-15s	KNN	0.9048	0.026	10	no
A2-VPN	E04-knn_log_standard-A2-VPN-15s	KNN	0.8544	0.0286	11	no
A2-NonVPN	E07-knn_log_standard-A2-NonVPN-15s	KNN	0.8929	0.0146	11	yes
B	E17-svm_rbf-B-15s	SVM-RBF	0.8242	0.0186	9	yes

The **comparison set** holds, per task, the best 15 s validation variant of each of the 7 method types, 28 models in all. It was fixed in `configs/final/compare/` before the test set is opened. In Phase 4 each is retrained on training + validation data and scored once on the test set, with paired bootstrap tests (Holm correction) for differences.

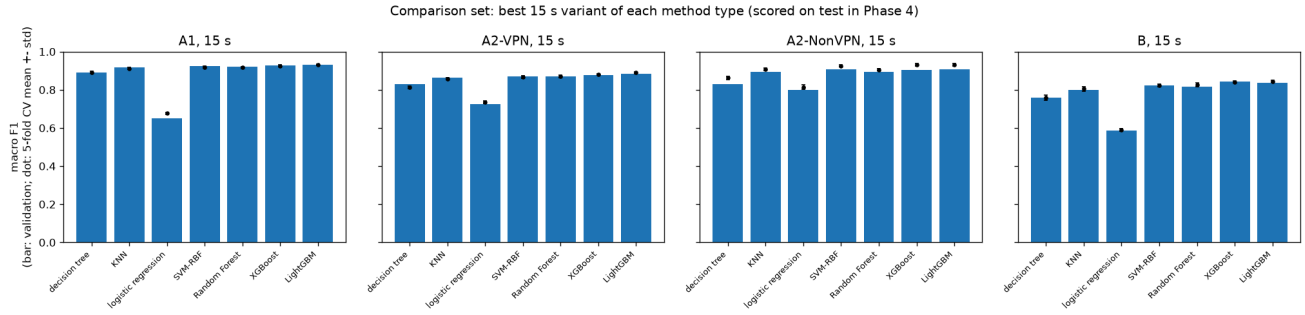


Figure 14. Comparison set at 15 s

task	method type	run	val macro F1	variants compared
A1	decision tree	E17-tree-A1-15s	0.8905	2
A1	KNN	E07-knn_log_standard-A1-15s	0.9163	4
A1	logistic regression	E17-logreg-A1-15s	0.6517	2
A1	SVM-RBF	E17-svm_rbf-A1-15s	0.9259	2
A1	Random Forest	E05-rf-A1-15s	0.9203	2
A1	XGBoost	E06-xgb-A1-15s	0.9271	3
A1	LightGBM	E17-lgbm-A1-15s	0.9308	2
A2-VPN	decision tree	E17-tree-A2-VPN-15s	0.8292	2
A2-VPN	KNN	E07-knn_log_standard-A2-VPN-15s	0.8625	4
A2-VPN	logistic regression	E17-logreg-A2-VPN-15s	0.7263	2
A2-VPN	SVM-RBF	E17-svm_rbf-A2-VPN-15s	0.8707	2
A2-VPN	Random Forest	E17-rf-A2-VPN-15s	0.8704	2
A2-VPN	XGBoost	E06-xgb-A2-VPN-15s	0.8758	2
A2-VPN	LightGBM	E08-lgbm_class_weight-A2-VPN-15s	0.883	4
A2-NonVPN	decision tree	E02-tree-A2-NonVPN-15s	0.8294	2
A2-NonVPN	KNN	E07-knn_log_standard-A2-NonVPN-15s	0.8929	4

task	method type	run	val macro F1	variants compared
A2-NonVPN	logistic regression	E17-logreg-A2-NonVPN-15s	0.8002	2
A2-NonVPN	SVM-RBF	E17-svm_rbf-A2-NonVPN-15s	0.9058	2
A2-NonVPN	Random Forest	E05-rf-A2-NonVPN-15s	0.8941	2
A2-NonVPN	XGBoost	E07-xgb-A2-NonVPN-15s	0.9025	2
A2-NonVPN	LightGBM	E08-lgbm_oversample-A2-NonVPN-15s	0.9075	5
B	decision tree	E02-tree-B-15s	0.7585	2
B	KNN	E07-knn_log_standard-B-15s	0.8006	4
B	logistic regression	E17-logreg-B-15s	0.5862	2
B	SVM-RBF	E17-svm_rbf-B-15s	0.8242	2
B	Random Forest	E17-rf-B-15s	0.8167	2
B	XGBoost	E07-xgb-B-15s	0.8428	4
B	LightGBM	E07-lgbm-B-15s	0.8352	2

4.14 E09: feature-group ablation (analysis only)

What. The four headline models retrained with one feature group only, with one group removed, and with the timing-only set (all but flowBytesPerSecond, the only size-based feature). **Threshold.** (a) every group alone scores below all 23 features in all four tasks; (b) dropping any one group costs at most 0.02 in at least 3 of 4 tasks; (c) the timing-only set is within 0.01 of all 23 features in at least 3 of 4 tasks. If any set scored at least 0.03 above all 23, the work would stop and the user would be asked (it did not happen).

task	feature set	features	val macro F1	change vs all	CV macro F1
A1	all	23	0.9308	0.0	0.9320 +- 0.0028
A1	only_fiat	4	0.8078	-0.1229	0.8111 +- 0.0059
A1	only_biat	4	0.8243	-0.1064	0.8136 +- 0.0100
A1	only_flowiat	4	0.8507	-0.08	0.8456 +- 0.0043
A1	only_active	4	0.5489	-0.3819	0.5618 +- 0.0124
A1	only_idle	4	0.5505	-0.3803	0.5614 +- 0.0107
A1	only_rates_duration	3	0.8392	-0.0915	0.8363 +- 0.0063
A1	no_fiat	19	0.9256	-0.0052	0.9187 +- 0.0050
A1	no_biat	19	0.9152	-0.0155	0.9161 +- 0.0050
A1	no_flowiat	19	0.9322	0.0014	0.9222 +- 0.0019
A1	no_active	19	0.9296	-0.0011	0.9300 +- 0.0024
A1	no_idle	19	0.9304	-0.0004	0.9302 +- 0.0031
A1	no_rates_duration	20	0.9185	-0.0122	0.9133 +- 0.0054
A1	timing_only	22	0.9159	-0.0148	0.9140 +- 0.0031
A2-VPN	all	23	0.883	0.0	0.8893 +- 0.0044
A2-VPN	only_fiat	4	0.7593	-0.1237	0.7326 +- 0.0146
A2-VPN	only_biat	4	0.7199	-0.1631	0.6958 +- 0.0108
A2-VPN	only_flowiat	4	0.7861	-0.0969	0.7690 +- 0.0107
A2-VPN	only_active	4	0.3252	-0.5578	0.3222 +- 0.0193
A2-VPN	only_idle	4	0.3067	-0.5763	0.3208 +- 0.0145
A2-VPN	only_rates_duration	3	0.779	-0.104	0.7661 +- 0.0082
A2-VPN	no_fiat	19	0.8651	-0.0179	0.8723 +- 0.0037

task	feature set	features	val macro F1	change vs all	CV macro F1
A2-VPN	no_biat	19	0.8724	-0.0105	0.8760 +- 0.0037
A2-VPN	no_flowiat	19	0.8752	-0.0078	0.8833 +- 0.0054
A2-VPN	no_active	19	0.886	0.003	0.8895 +- 0.0041
A2-VPN	no_idle	19	0.8826	-0.0004	0.8893 +- 0.0054
A2-VPN	no_rates_duration	20	0.8435	-0.0395	0.8449 +- 0.0083
A2-VPN	timing_only	22	0.849	-0.034	0.8410 +- 0.0095
A2-NonVPN	all	23	0.9075	0.0	0.9310 +- 0.0058
A2-NonVPN	only_fiat	4	0.7535	-0.154	0.7720 +- 0.0087
A2-NonVPN	only_biat	4	0.7271	-0.1804	0.7407 +- 0.0140
A2-NonVPN	only_flowiat	4	0.8165	-0.091	0.8288 +- 0.0095
A2-NonVPN	only_active	4	0.3355	-0.572	0.3208 +- 0.0128
A2-NonVPN	only_idle	4	0.3038	-0.6037	0.3225 +- 0.0177
A2-NonVPN	only_rates_duration	3	0.8471	-0.0604	0.8396 +- 0.0110
A2-NonVPN	no_fiat	19	0.9024	-0.0051	0.9260 +- 0.0087
A2-NonVPN	no_biat	19	0.8993	-0.0082	0.9240 +- 0.0087
A2-NonVPN	no_flowiat	19	0.903	-0.0045	0.9189 +- 0.0065
A2-NonVPN	no_active	19	0.9093	0.0018	0.9283 +- 0.0066
A2-NonVPN	no_idle	19	0.9039	-0.0036	0.9300 +- 0.0045
A2-NonVPN	no_rates_duration	20	0.8774	-0.0301	0.8980 +- 0.0037
A2-NonVPN	timing_only	22	0.8744	-0.0331	0.9000 +- 0.0060
B	all	23	0.8428	0.0	0.8405 +- 0.0056
B	only_fiat	4	0.64	-0.2028	0.6399 +- 0.0089
B	only_biat	4	0.6098	-0.233	0.6147 +- 0.0121
B	only_flowiat	4	0.6716	-0.1712	0.6926 +- 0.0102
B	only_active	4	0.2138	-0.629	0.2262 +- 0.0072
B	only_idle	4	0.2083	-0.6345	0.2118 +- 0.0092
B	only_rates_duration	3	0.7043	-0.1385	0.6886 +- 0.0091
B	no_fiat	19	0.8244	-0.0184	0.8266 +- 0.0072
B	no_biat	19	0.8163	-0.0265	0.8189 +- 0.0067
B	no_flowiat	19	0.8262	-0.0166	0.8284 +- 0.0052
B	no_active	19	0.8479	0.0051	0.8400 +- 0.0071
B	no_idle	19	0.8428	0.0	0.8405 +- 0.0058
B	no_rates_duration	20	0.7879	-0.0549	0.7936 +- 0.0076
B	timing_only	22	0.7987	-0.0441	0.7949 +- 0.0062

Result. No single group comes close to all 23 features (the best single group is 0.0604-0.1385 below). Rates + duration matter most: removing them costs 0.0122-0.0549. Active or idle features can be removed with almost no effect (at most +-0.0051). The timing-only set loses 0.0148-0.0441, so packet sizes carry information here.
Verdict. (a) supported, (b) not supported, (c) not supported. Nothing changes in the final configurations. [E09]

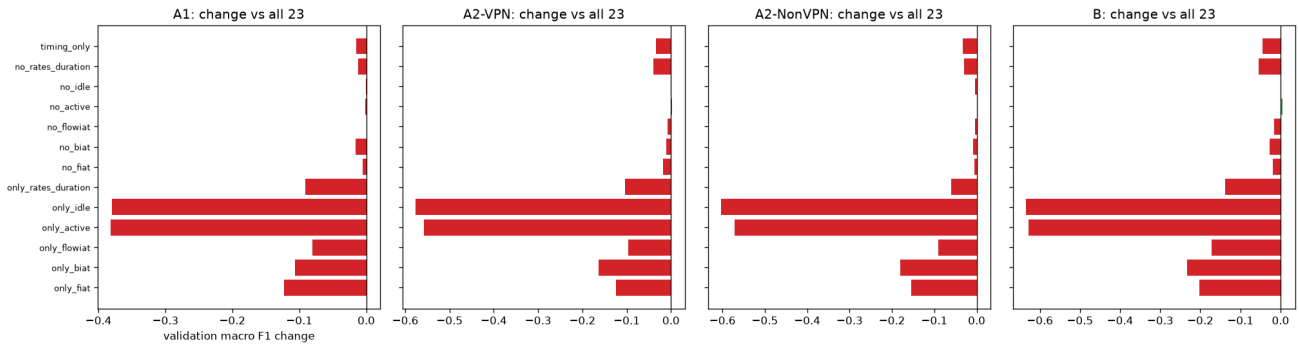


Figure 15. E09 feature-group ablation

4.15 E13: learning curve (analysis only)

What. The headline models trained on 10%, 20%, 40%, 60% and 80% of the training rows (stratified samples, 5 seeds each) and scored on the full validation set; 100% is the headline run. **Threshold.** (a) 100% scores above 10% in all four tasks; (b) the gain from 80% to 100% is below 0.01 in at least 3 of 4 tasks; (c) the spread over seeds is larger at 10% than at 80% in at least 3 of 4 tasks.

task	train fraction	runs	training rows	val macro F1 mean	val macro F1 std
A1	0.1	5	1,262	0.8724	0.0062
A1	0.2	5	2,525	0.8952	0.0041
A1	0.4	5	5,050	0.9106	0.0066
A1	0.6	5	7,575	0.9213	0.0039
A1	0.8	5	10,100	0.927	0.0029
A1	1.0	1	12,626	0.9308	
A2-VPN	0.1	5	656	0.7793	0.0052
A2-VPN	0.2	5	1,313	0.826	0.0101
A2-VPN	0.4	5	2,626	0.8582	0.0041
A2-VPN	0.6	5	3,939	0.8721	0.0053
A2-VPN	0.8	5	5,252	0.8804	0.0036
A2-VPN	1.0	1	6,566	0.883	
A2-NonVPN	0.1	5	608	0.8176	0.0093
A2-NonVPN	0.2	5	1,217	0.8585	0.0118
A2-NonVPN	0.4	5	2,434	0.8806	0.0057
A2-NonVPN	0.6	5	3,651	0.8934	0.0045
A2-NonVPN	0.8	5	4,868	0.9005	0.0033
A2-NonVPN	1.0	1	6,085	0.9075	
B	0.1	5	1,265	0.6969	0.0097
B	0.2	5	2,530	0.7468	0.0138
B	0.4	5	5,060	0.7975	0.0036
B	0.6	5	7,590	0.8156	0.0016
B	0.8	5	10,120	0.8297	0.0029
B	1.0	1	12,651	0.8428	

Result. Every task gains with more data (10% to 100%: +0.0584 to +0.1459). A1 and both A2 tasks are near their plateau (80% to 100%: +0.0026 to +0.0070), but the 14-class task B still gains +0.0131. **Verdict.** (a), (b) and (c) supported. [E13]

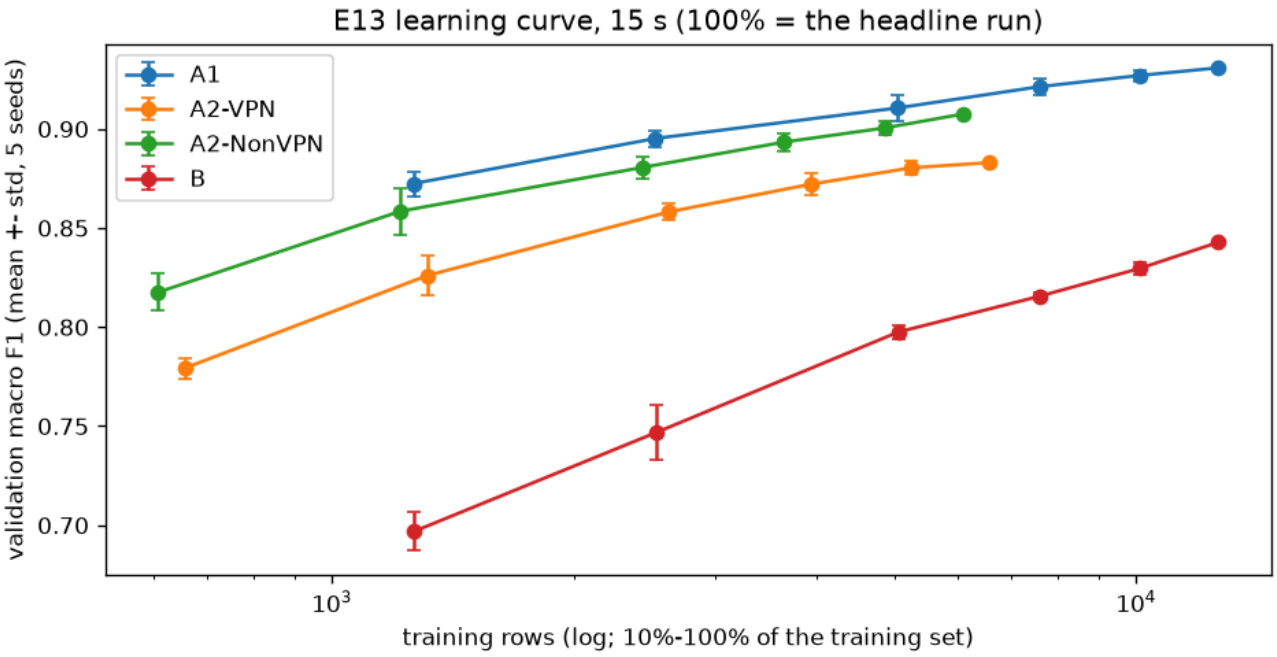


Figure 16. E13 learning curve

4.16 E14: split-seed stability (analysis only)

What. The headline models retrained on 10 new stratified splits of training + validation (seeds 42-51, same sizes). The seed-42 test rows are never pooled or looked at. Seed 42 here is a resplit, not the official split. **Threshold.** (a) the standard deviation is below 0.01 in at least 3 of 4 tasks; (b) mean cross-validation minus mean validation is within 0.01 in at least 3 of 4 tasks; (c) the official validation score lies inside the range of the 10 resplits in all four tasks.

task	seeds	val macro F1 mean	val macro F1 std	val macro F1 min	val macro F1 max	val accuracy mean	val accuracy std	CV macro F1 mean	CV - val mean	official split (seed 42) val macro F1
A1	10	0.9338	0.0042	0.9275	0.9397	0.9339	0.0042	0.9306	-0.0032	0.9308
A2-VPN	10	0.8862	0.0114	0.8689	0.902	0.9037	0.0066	0.8805	-0.0057	0.883
A2-NonVPN	10	0.9324	0.0071	0.9257	0.9468	0.9589	0.0044	0.9283	-0.0041	0.9075
B	10	0.8476	0.0084	0.8344	0.8636	0.8819	0.0074	0.8358	-0.0119	0.8428

Result. Scores move with the split by a standard deviation of 0.0042-0.0114. In A2-NonVPN the official validation score (0.9075) is below all 10 resplits (0.9257-0.9468): its validation rows are unusually hard, mostly for MAIL and P2P. This also explains the large cross-validation minus validation gaps seen for A2-NonVPN in E04-E17. **Verdict.** (a) supported, (b) supported, (c) not supported. [E14]

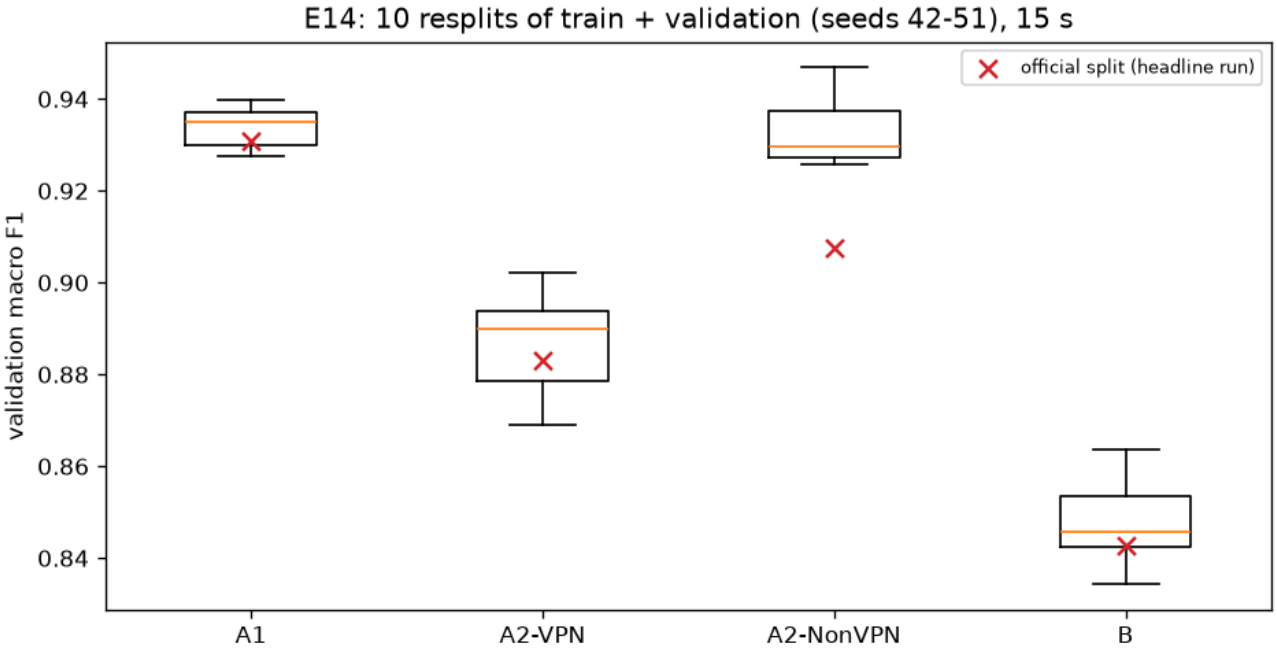


Figure 17. E14 seed stability

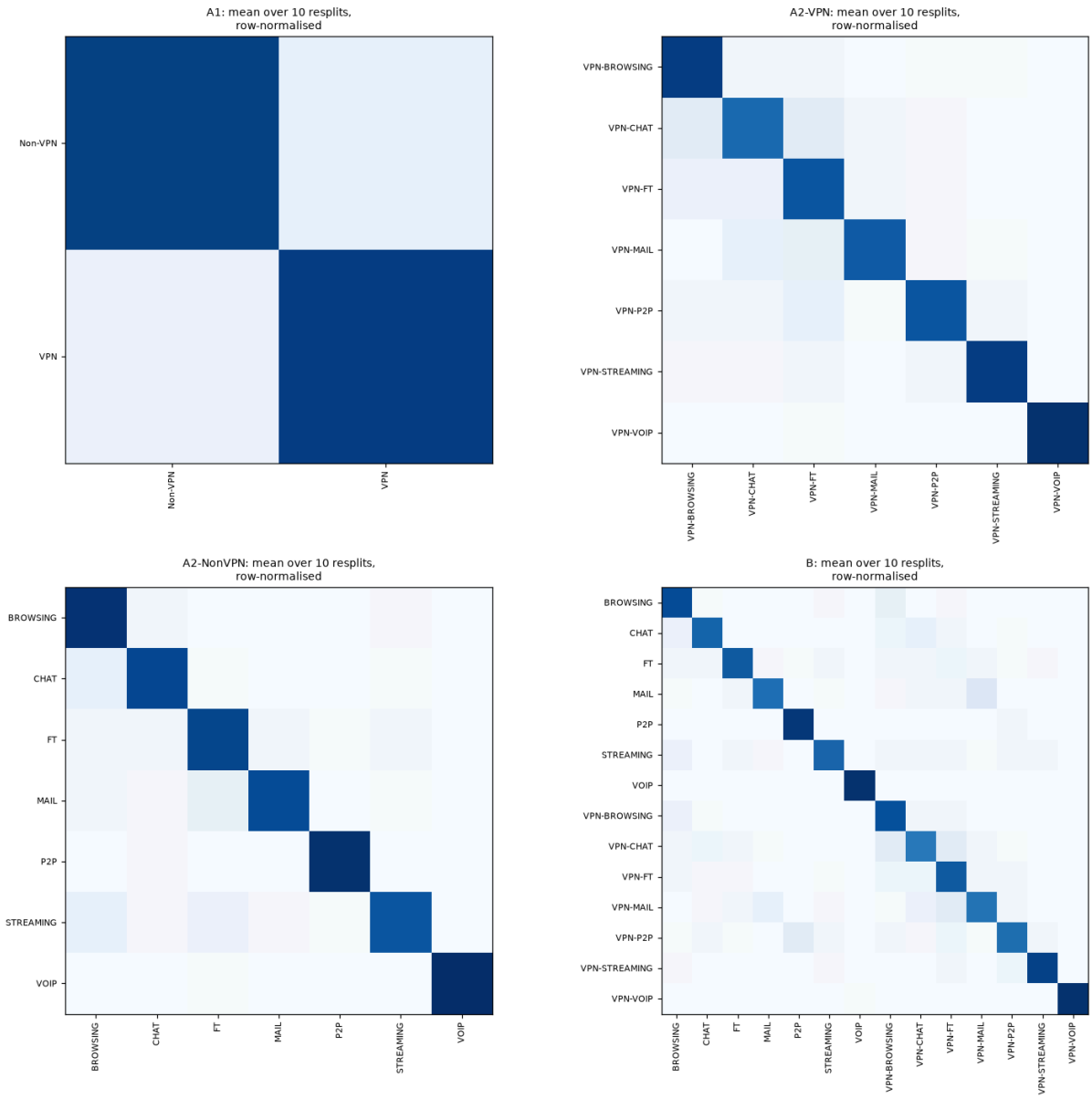


Figure 18. E14 mean confusion matrices

4.17 E11: session-aware split (not run)

The ARFF files hold only the 23 features and the label, with no flow IDs, addresses, ports, timestamps or capture-file names, so flows cannot be grouped by recording session (E00). E11 therefore does not run. Exact duplicates were removed before the split (E00), and E14 and E16(b) measure related effects. The missing session split is a limitation (Section 7). [E11]

4.18 E16: reproducing Isnan et al. (2026), then correcting it

What. Isnan et al. report 88.01% accuracy for a 300-tree Random Forest on the Kaggle copy of the dataset, which is the eight A2 files from all four timeouts stacked (Section 2.9). E16 used its own data version (kaggle-k1) and ran four parts:

- **(a)** their exact setup: duplicates dropped over the whole file, a stratified 80/20 split with seed 42, and their grid search with accuracy as the score;
- **(b)** the same method with each timeout kept apart;
- **(c)** our tuned task-B XGBoost under (b);
- **(d)**, added in a recheck: (a)'s forest trained on mixed-timeout rows cut to each per-timeout training size.

Threshold. (a) accuracy and weighted F1 within 0.01 of theirs; (b) the per-timeout mean is at least 0.01 below (a); (c) our model's per-timeout mean is at least the forest's; (d) at equal training size, mixing timeouts adds less than 0.01 over (b).

run	split	train / test rows	grid pick	grid best CV accuracy	test accuracy	test weighted F1	test macro F1
E16-isnan-rf	isnan	32,789 / 8,198	{"max_depth": null, "min_samples_leaf": 1, "min_samples_split": 2, "n_estimators": 200}	0.8742	0.8796	0.8792	0.8407
E16-per_timeout-rf-120s	per_timeout_120s	7,671 / 1,918	{"max_depth": null, "min_samples_leaf": 1, "min_samples_split": 5, "n_estimators": 300}	0.8178	0.8206	0.817	0.7724
E16-per_timeout-rf-15s	per_timeout_15s	14,459 / 3,615	{"max_depth": 30, "min_samples_leaf": 1, "min_samples_split": 2, "n_estimators": 300}	0.8578	0.8658	0.8652	0.8323
E16-per_timeout-rf-30s	per_timeout_30s	10,910 / 2,728	{"max_depth": 30, "min_samples_leaf": 1, "min_samples_split": 2, "n_estimators": 300}	0.8457	0.8537	0.8527	0.815
E16-per_timeout-rf-60s	per_timeout_60s	11,783 / 2,946	{"max_depth": 30, "min_samples_leaf": 1, "min_samples_split": 2, "n_estimators": 300}	0.8307	0.8316	0.8308	0.7873
E16-per_timeout-xgb-120s	per_timeout_120s	7,671 / 1,918	fixed (configs/final/B.yaml)		0.8384	0.8346	0.7881
E16-per_timeout-xgb-15s	per_timeout_15s	14,459 / 3,615	fixed (configs/final/B.yaml)		0.8924	0.8919	0.8637
E16-per_timeout-xgb-30s	per_timeout_30s	10,910 / 2,728	fixed (configs/final/B.yaml)		0.8728	0.872	0.8327
E16-per_timeout-xgb-60s	per_timeout_60s	11,783 / 2,946	fixed (configs/final/B.yaml)		0.8551	0.8544	0.8076

training rows (timeouts mixed)	runs	test accuracy mean	std	min - max	15s test rows	30s test rows	60s test rows	120s test rows	(b) same rows, one timeout
32,789	1	0.8796			0.8913	0.9038	0.8982	0.8222	
14,459	5	0.8504	0.0021	0.8470 - 0.8529	0.8643	0.8798	0.8655	0.7887	15s: 0.8658
11,783	5	0.8435	0.0028	0.8401 - 0.8464	0.8592	0.8742	0.8558	0.7809	60s: 0.8316
10,910	5	0.8396	0.0024	0.8367 - 0.8430	0.8559	0.8708	0.8517	0.7755	30s: 0.8537
7,671	5	0.8243	0.0038	0.8202 - 0.8302	0.8402	0.8576	0.8372	0.7583	120s: 0.8206

Result.

- **(a)** Their result is reproduced: 0.8796 accuracy and 0.8792 weighted F1, against their 0.8801 and 0.8797, on the same split. Their accuracy hides weaker small classes (macro F1 0.8407).
- **(b)** With the timeouts kept apart, the same method scores 0.8206-0.8658 (mean 0.8430).
- **(d)** That drop is explained by training-set size, not by the mixing. Stacking the four files gives 2.3-4.3 times as many training rows. At equal size the mixed setup scores 0.8395 on average, slightly below (b).
- **(c)** Our tuned XGBoost beats their forest at every timeout (mean 0.8647; 0.8924 at 15 s).

The Kaggle rows use the A2 column layout and an 80/20 split, so these scores are not comparable with our validation scores. **Verdict.** (a), (b), (c) and (d) supported. E16 feeds no model selection. [E16]

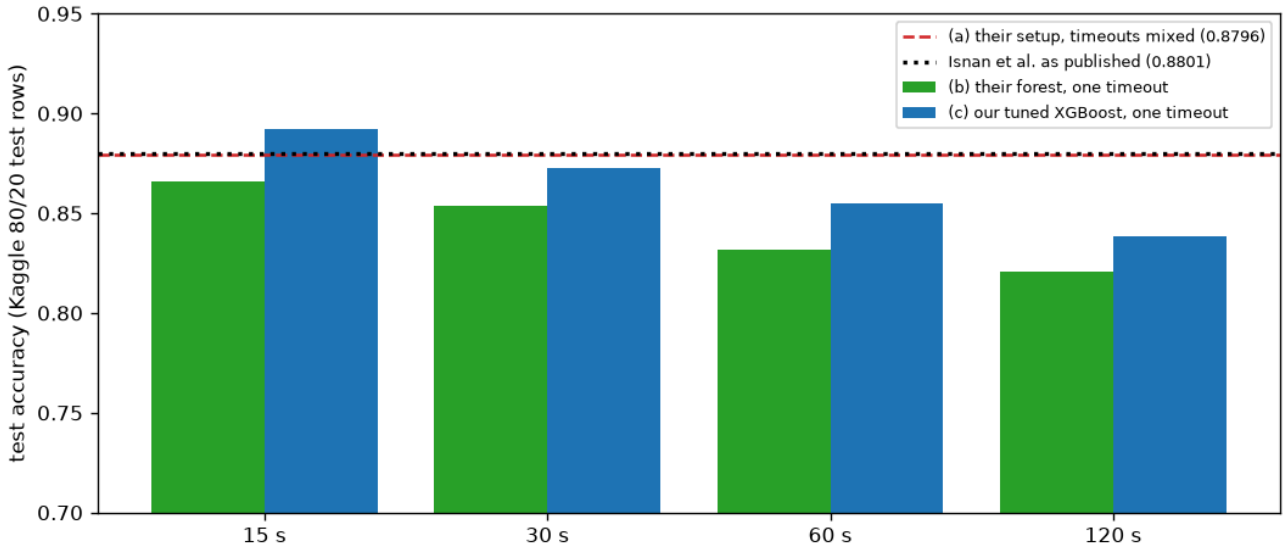


Figure 19. E16 Isnan et al. reproduced and corrected

4.19 Storage after Phase 3

Every official run saves its model (compressed, level 3). Per experiment and model type:

kind	runs	total_MB	per_run_max_MB	model_file_max_MB
E04 knn_log_robust	4	4.678	1.577	1.435
E04 knn_log_standard	7	6.54	1.579	1.437
E05 rf	7	102.403	22.6	22.436
E06 lgbm	4	4.56	2.168	2.014
E06 xgb	4	2.522	1.226	1.064
E07 knn_log_standard	4	4.78	1.604	1.437
E07 lgbm	3	92.646	62.694	62.51
E07 rf	1	3.835	3.835	3.72
E07 xgb	4	9.238	4.508	4.315
E08 lgbm	5	67.775	27.135	27.033
E08 xgb	2	4.724	4.511	4.348
E09 lgbm	39	482.749	27.644	27.549
E09 xgb	13	61.881	5.261	5.092
E10 xgb_two_step	1	3.856	3.856	3.728
E13 lgbm	75	474.7	21.265	21.163
E13 xgb	25	73.528	4.188	4.025
E14 lgbm	30	372.146	25.859	25.759
E14 xgb	10	44.771	4.494	4.336
E16 rf	26	453.605	35.049	34.902
E16 xgb	4	17.765	4.829	4.697
E17 lgbm	1	7.215	7.215	7.092
E17 logreg	4	0.515	0.199	0.004
E17 rf	3	68.011	45.962	45.773
E17 svm_rbf	4	2.944	1.479	1.306

kind	runs	total_MB	per_run_max_MB	model_file_max_MB
E17 tree	4	0.732	0.275	0.07

Whole store at this point: artifacts 2645.3 MiB, mlflow.db 35.8 MiB. This is far below the plan's 20 GB limit. The four headline model files are 4.31-25.12 MiB, all below GitHub's 50 MB warning size.

References

1. G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Encrypted and VPN Traffic using Time-related Features," in *Proc. 2nd Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2016, pp. 407–414.
2. Canadian Institute for Cybersecurity, University of New Brunswick, "VPN-nonVPN dataset (ISCXVPN2016)." [Online]. Available: https://www.unb.ca/cic/datasets/vpn.html
3. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
4. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
5. G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems 30*, 2017.
6. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30*, 2017.
7. T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proc. 25th ACM SIGKDD*, 2019, pp. 2623–2631.
8. M. Zaharia et al., "Accelerating the Machine Learning Lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.
9. G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," *JMLR*, vol. 18, no. 17, pp. 1–5, 2017.
10. N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
11. J. R. Quinlan, *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.
12. S. Bagui, X. Fang, E. Kalaimannan, S. C. Bagui, and J. Sheehan, "Comparison of machine-learning algorithms for classification of VPN network traffic flow using time-related features," *Journal of Cyber Security Technology*, vol. 1, no. 2, pp. 108–126, 2017, doi: 10.1080/23742917.2017.1321891.
13. W. Isnan, Y. Fatmi, R. Darni, and V. Aulia, "Flow-Based Encrypted Network Traffic Classification Using Random Forest for Network Access Control," *Intechno Journal: Information Technology Journal*, vol. 8, no. 1, 2026, doi: 10.24076/intechnojournal.2026v8i1.2922.
14. S. Jorgensen et al., "Extensible Machine Learning for Encrypted Network Traffic Application Labeling via Uncertainty Quantification," *IEEE Transactions on Artificial Intelligence*, 2023, doi: 10.1109/TAI.2023.3244168 (arXiv:2205.05628).
15. G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Encrypted and VPN Traffic Using Time-Related Features" (poster), UNB Research Expo, 2016. [Online]. Available: https://www.cs.unb.ca/research-expo/expos/2016/
16. University of Waikato, "Weka 3 API documentation (stable)": classes `weka.classifiers.trees.J48`, `weka.classifiers.lazy.IBk`, `weka.core.EuclideanDistance` and `weka.core.NormalizableDistance`. [Online]. Available: https://weka.sourceforge.io/doc.stable/ (accessed 28 Oct 2026).